Automotive Engineering Honors Mini Project Titled

OBSTACLE DETECTION AND AUTOMATED STEERING USING PYTHON AND MATLAB SIMULINK

Submitted in partial fulfilment of
the requirement for the Honors Course of
Automotive Engineering under
Degree of Bachelor of Technology (B.Tech.),
In
Mechanical Engineering

By

Grishma Balgi (211021009)

Dev Ganatra (211020018)



Prof. H. P. Khairnar

Guide

Mechanical Department

Veermata Jijabai Technological Institute

Mumbai 400 019

2024-25

CERTIFICATE

The report "**OBSTACLE DETECTION AND AUTOMATED STEERING USING PYTHON AND MATLAB SIMULINK**" submitted by Grishma Balgi & Dev Ganatra is found to be satisfactory and is approved for the degree of B.Tech in Mechanical Engineering (Hons. Automotive Engineering).

Dr. H. P. Khairnar
Guide

Examiner

Dr. W. S. Rathod
Head of Department
Mechanical Engineering

Date:

Place: V.J.T.I., Mumbai

ACKNOWLEDGMENT

I would like to express my sincere gratitude to the following individuals who have played a pivotal role in the successful completion of my project and the preparation of this report:

I am fortunate to have had the opportunity to work with such a dedicated and supportive team at VJTI. Their guidance and mentorship has been instrumental in shaping my career. I have gained invaluable experience and knowledge that will undoubtedly serve me well in the future.

A special thanks to **Dr. H.P. Khairnar**, Professor at VJTI, for his unwavering belief in my potential. His mentorship and leadership have been a source of inspiration throughout my project. **Dr. Wankhade's** guidance and support have been instrumental in shaping my academic journey.

Thank you once again for your unwavering support and encouragement.

Grishma Balgi **Dev Ganatra**

(211021009) **(211020018)**

DECLARATION

I declare that this written submission represents my ideas in my own words and where other's ideas or words have been included, I have adequately cited and referenced the original sources.

I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea / data / fact / source in my submission.

I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Grishma Balgi

(211021009)

Dev Ganatra

(211020018)

Date:

ABSTRACT

This project presents a robust simulation framework that integrates sensor emulation, machine learning-based decision logic, and dynamic vehicle control for autonomous navigation. With the rapid evolution of intelligent transportation systems, the ability of autonomous vehicles to detect obstacles and steer accordingly has become essential. This work replicates such functionalities in a simulated multi-lane environment, leveraging MATLAB Simulink for modeling and Python for predictive analytics.

A virtual car is designed to traverse a three-lane road populated with randomly appearing obstacles. The simulation replicates a LiDAR-like sensor that continuously calculates the relative distance between the vehicle and potential obstacles. When an object is detected within a critical threshold, a trained machine learning model—specifically a Decision Tree Classifier—predicts the likelihood of collision. Based on this prediction, the system executes a controlled steering maneuver to shift the vehicle into a safer lane. Smooth transitions are ensured by incorporating velocity reduction and gradual lateral displacement to emulate realistic driving dynamics.

To facilitate real-time interactivity, the Python model is integrated with MATLAB using the pyenv interface. A comprehensive dataset was generated using MATLAB simulations and exported as CSV for model training. The Decision Tree achieved a high accuracy, validating its suitability for real-time prediction. Additionally, a dynamic graphical user interface (GUI) was developed in MATLAB to visualize the car's motion, lane transitions, and obstacle avoidance behavior. The GUI offers both animation and decision logic visualization, making the project not only technically sound but also intuitively demonstrable.

This mini project successfully demonstrates how simulation, data science, and system integration can come together to model real-world autonomous vehicle behavior. It provides a solid foundation for further exploration in advanced driver assistance systems (ADAS) and autonomous control systems. The approach outlined herein is modular, interpretable, and scalable, making it highly adaptable for future research and deployment.

Keywords: Obstacle, Detection, Python, MATLAB, Simulink, Steering

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION..... 1

1.1 Objectives..... 2

1.2 Report Organization..... 2

CHAPTER 2: LITERATURE REVIEW..... 4

2.1 Autonomous Driving 4

2.2 Obstacle Detection 5

2.3 Automated Steering 6

2.4 MATLAB Simulink 7

2.5 Machine Learning with Python 10

2.5.1 Supervised Learning Algorithms..... 10

2.5.2 Unsupervised Learning..... 11

2.5.3 Reinforcement Learning..... 12

2.6 Review of Previously Published Work..... 12

2.7 Summary of Literature Review..... 14

CHAPTER 3: MODEL DEVELOPMENT..... 16

3.1 Simulink Model..... 16

3.1.1 Vehicle Motion in X-Direction..... 16

3.1.2 Obstacle Placement and Distance Measurement..... 17

3.1.3 Collision Detection Logic..... 18

3.1.4 Steering Logic and Lane Change Behavior..... 19

3.1.5 Output Parameters..... 19

3.1.6 Visualization with XY Graph..... 20

3.2 Data Collection using MATLAB Code..... 21

3.3 Python Code....24
3.3.1 Dataset Preparation and Feature Engineering....25
3.3.2 Model Selection and Training....25
3.3.3 Model Serialization and MATLAB Interfacing....26
3.3.4 Model Visualization and Explainability....26
3.3.5 Real-Time Prediction Function for Simulation Integration....26
3.4 Python to MATLAB....27
3.4.1 Integration of Python with MATLAB....29
3.4.2 Simulation Design and Obstacle Management....30
3.4.3 Lane Changing Logic and Animation....31
CHAPTER 4: RESULTS AND DISCUSSIONS....33
4.1 Obstacle Detecting Simulator....33
4.1.1 Output 1 – Collision Detection Scope Signal....34
4.1.2 Output 2 – XY Graph Trajectory Visualization....35
4.2 Data as Comma Separated Values....37
4.3 Decision Tree....39
4.3.1 Dataset and Feature Selection....40
4.3.2 Model Training and Accuracy....41
4.3.3 Visualization and Interpretation of the Tree....41
4.3.4 Integration into the Lane Change Decision Logic....42
4.3.5 Discussion on Model Robustness and Observations....43
4.4 Graphical User Interface....43
4.4.1 Objective of GUI Simulation....44
4.4.2 Working Principle of the Simulation....45
4.4.3 Case 1: Lane Change from Bottom Lane to Middle Lane....46
4.4.4 Case 2: Lane Change from Middle Lane to Bottom Lane....48
4.4.5 Case 3: Lane Change from Middle Lane to Top Lane....50

4.4.6 Case 4: Lane Change from Top Lane to Middle Lane..... 53

CHAPTER 5: CONCLUSION & FUTURE SCOPES.....56

5.1 Conclusion & Summary.....56

5.2 Future Scope..... 57

***REFERENCES*.....59**

LIST OF FIGURES

Figure 1. Tesla Model 3 5

Figure 2. LiDAR Sensor Working Depiction 6

Figure 3. Block diagram of classification methodology for object-type identification..... 14

Figure 4. Simulink Model..... 16

Figure 5. Vehicle Motion in X-Direction..... 17

Figure 6. Obstacle Placement and Distance Measurement..... 18

Figure 7. Collision Detection Logic..... 18

Figure 8. Steering Logic and Lane Change Behavior..... 19

Figure 9. Output Parameters..... 20

Figure 10. Data Collection Using MATLAB..... 21

Figure 11. Model Training(Python) Using Decision Tree..... 24

Figure 12. Python-MATLAB Integration-I……..……25

Figure 13. Python-MATLAB Integration - II..... 28

Figure 14. Python-MATLAB Integration-III……..……..28

Figure 15. Python-MATLAB Integration - IV..... 29

Figure 16. Python-MATLAB Integration - V..... 29

Figure 17. Scope Result of Simulink Model..... 34

Figure 18. XY Plot Result of Simulink Model..... 35

Figure 19. Comma Separated Values File Snippet..... 37

Figure 21. Decision Tree..... 40

Figure 22. Car running safe in the bottom lane just before the obstacle……46

Figure 23. Car changing lane from the bottom lane towards middle lane..... 46

Figure 24. Car running in Middle after lane change..... 47

Figure 25. Car running safe in the midline lane just before the obstacle……..49

Figure 26. Car changing lane from middle lane towards bottom lane……..…49

Figure 27. Car running in the bottom lane after lane change..... 49

Figure 28. Car running in the middle lane just before obstacle…….……...51

Figure 29. Car changing lane from the middle lane towards top lane..... 51
Figure 30. Car running in the top lane after lane change.....51
Figure 31. Car running in the top lane just before obstacle…….……....52
Figure 32. Car changing lane from the top lane towards middle lane..... 52
Figure 33. Car running in the middle lane after lane change.....52

CHAPTER 1: INTRODUCTION

Autonomous vehicles are rapidly transforming the landscape of transportation, promising safer roads, reduced human error, and enhanced mobility solutions. Central to the development of such intelligent vehicles are systems that can perceive their surroundings, interpret the data, and make decisions in real time. The present project, "Obstacle Detection and Autonomous Steering using Python (Machine Learning) and MATLAB Simulink," aims to simulate these core functionalities in a controlled virtual environment, providing a robust prototype for understanding and testing autonomous vehicle behavior.

Obstacle detection is a fundamental requirement in any autonomous system, enabling the vehicle to recognize, classify, and respond to physical objects within its vicinity. In real-world scenarios, this is achieved using various sensors such as LiDAR, radar, or ultrasonic sensors. For this project, virtual sensors are simulated within MATLAB Simulink to emulate the role of these physical components, capturing data such as object position, relative distance, and car velocity. This data serves as the sensory input that mimics a real vehicle's perception system.

The concept of automated steering refers to the vehicle’s ability to make intelligent decisions regarding its direction of motion. In this simulation, the system dynamically alters the car’s trajectory based on predicted obstacles ahead. Steering decisions are not hardcoded; instead, they are driven by machine learning predictions, offering a data-driven approach to maneuvering. By automating the decision-making process, the system reflects how autonomous vehicles adjust their paths to ensure collision-free navigation.

The simulation environment was developed using MATLAB Simulink, a widely used platform for modeling, simulating, and analyzing dynamic systems. Simulink allows real-time representation of a car navigating a multi-lane road with randomly positioned obstacles. The motion of the car and obstacles is updated continuously, enabling detailed visualization of avoidance behavior, trajectory change, and lane-switching logic.

A critical aspect of this project is the integration of MATLAB and Python, which bridges the gap between simulation and predictive modeling. This interface allows MATLAB to execute Python scripts during runtime, enabling seamless communication and real-time feedback between the simulation and the machine learning model. Such integration replicates real-world automotive systems, where various subsystems need to interact and operate synchronously. The successful execution of this project lays the groundwork for more complex autonomous behaviors and highlights the potential of combining simulation platforms with machine learning for rapid prototyping and validation of intelligent vehicular systems.

1.1 Objectives

The primary aim of this project is to develop a simulation-based system that demonstrates intelligent obstacle detection and autonomous steering using machine learning.. By combining the capabilities of MATLAB Simulink for dynamic simulation and Python for predictive modeling, the project seeks to emulate the behavior of an autonomous vehicle navigating a road with multiple lanes and randomly appearing obstacles. The objective is to enable the simulated car to detect obstacles in real time, predict the likelihood of collision, and make autonomous decisions to steer away safely, replicating real-world driving scenarios.

1.2 Report Organization

This report is organized in a logical and progressive manner, beginning with an introduction to the problem space and gradually moving toward system development, evaluation, and conclusions. The initial section introduces the motivation behind the project, highlighting the importance of obstacle detection and autonomous steering in the realm of intelligent transportation systems. It outlines the primary objectives of the work and presents a brief overview of the chosen tools and methodologies.

Following this, the literature review delves into the key technologies that form the foundation of the project. It explores MATLAB Simulink as a dynamic system modeling platform and Python as a versatile tool for implementing machine learning algorithms. Several related works are examined to identify gaps, trends, and relevant applications in the field of autonomous navigation.

The next section describes the detailed methodology used to develop the system. It begins with the creation of a Simulink model that simulates obstacle detection and decision-making logic. It also explains how time-series data was collected under various input conditions and how this data was processed to train a Decision Tree model in Python. The section concludes by discussing the integration of Python logic back into the MATLAB environment to enable real-time predictions and steering responses/

The final section is devoted to results and discussion. It presents the behavior of the vehicle under simulated conditions, evaluates model performance, and showcases the graphical interface that visualizes lane changes and obstacle avoidance in real time. The report concludes with key takeaways and potential areas for future development.

CHAPTER 2: LITERATURE REVIEW

2.1 Autonomous Driving [1]

Autonomous driving refers to the ability of a vehicle to operate without direct human input by perceiving its environment and making real-time navigation decisions. This capability arises from the integration of sophisticated systems including perception, localization, planning, and control. Perception systems rely on multi-modal sensor arrays—such as LiDAR, radar, cameras, and ultrasonic sensors—to detect static and dynamic elements in the environment, including lane markings, traffic signs, pedestrians, and other vehicles. Sensor fusion algorithms synthesize the data from these inputs to construct a real-time, high-fidelity representation of the surrounding environment. Localization methods, often aided by GPS and inertial measurement units (IMUs), determine the vehicle's precise position within a mapped environment. Path planning and control subsystems then process this situational data to generate and execute safe, smooth trajectories.

The automotive engineering community classifies autonomous driving capabilities using the Society of Automotive Engineers (SAE) J3016 standard, which defines six levels of driving automation. Level 0 implies no automation, while Levels 1 and 2 introduce driver assistance features like lane keeping and adaptive cruise control. Level 3 enables conditional automation, where the vehicle can handle dynamic driving tasks but requires human intervention upon request. Level 4 vehicles are capable of full autonomy within geofenced areas, requiring no human input under specific conditions. Level 5 represents full automation in all scenarios, eliminating the need for steering wheels or pedals. Tesla's Autopilot and Full Self-Driving (FSD) features currently align with Levels 2 and partially 3, offering lane centering, automatic lane changes, and traffic-aware cruise control. Tesla utilizes a vision-centric neural network approach, processing high-resolution visual data collected from its fleet to improve its AI models iteratively.

The development of fully autonomous vehicles represents a complex, multidisciplinary challenge, integrating control theory, robotics, embedded systems, and AI. As regulatory

frameworks evolve and sensor technologies mature, the path toward higher levels of autonomy continues to advance rapidly.



Figure 1. Tesla Model 3 [2]

2.2 Obstacle Detection [3]

Obstacle detection is a fundamental aspect of autonomous driving and advanced driver assistance systems (ADAS), serving as the primary input mechanism for dynamic environment perception and collision avoidance. It involves the identification, localization, and classification of objects in the vehicle's surrounding environment using a combination of sensory inputs and computational algorithms. The core objective of obstacle detection systems is to provide real-time information about the presence and trajectory of both static (e.g., roadblocks, traffic cones, curbs) and dynamic (e.g., pedestrians, cyclists, vehicles) hazards on or near the vehicle's path.

To achieve robust and accurate obstacle detection, modern autonomous systems utilize a suite of heterogeneous sensors, including LiDAR (Light Detection and Ranging), millimeter-wave radar, ultrasonic sensors, and vision-based systems (monocular, stereo, and fisheye cameras). LiDAR provides high-resolution 3D point clouds with precise range and angular data, making it particularly effective for object boundary estimation and depth perception. Radar systems

offer superior performance under adverse weather conditions and are capable of detecting the relative velocity of surrounding objects using the Doppler effect.

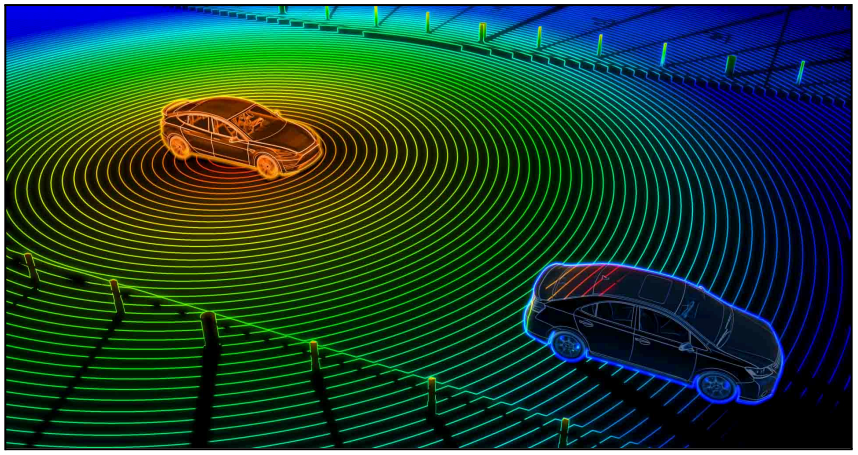


Figure 2. LiDAR Sensor Working Depiction [4]

The fusion of sensor data using probabilistic frameworks—such as Kalman filtering, Bayesian occupancy grids, and deep learning-based fusion—enhances the robustness of detection under varying environmental conditions. Algorithms then process this multi-sensor data to identify obstacles, predict motion trajectories, and assign threat levels, forming the basis for downstream control actions such as lane changes, emergency braking, or evasive maneuvers.

Accurate and timely obstacle detection is critical for maintaining situational awareness, enabling safe path planning, and supporting decision-making in real-time traffic scenarios. Continuous advancements in sensor hardware, perception algorithms, and real-time computing architectures are driving significant improvements in the safety and reliability of obstacle-aware autonomous systems.

2.3 Automated Steering [5]

Automated steering refers to the capability of a vehicle to dynamically alter its steering angle and trajectory in response to environmental inputs, sensor data, and control commands without human intervention. It constitutes a critical component of the lateral control system

within autonomous and semi-autonomous vehicles. Unlike traditional manual steering, automated steering relies on electromechanical actuators and advanced control algorithms to ensure that the vehicle maintains lane discipline, performs safe lane changes, and avoids detected obstacles while preserving ride stability and passenger comfort.

At its core, an automated steering system integrates real-time data from perception modules—such as obstacle detection, lane marking recognition, and vehicle localization—into motion planning and control architectures. The control module, often based on Proportional-Integral-Derivative (PID), Linear Quadratic Regulator (LQR), or Model Predictive Control (MPC) algorithms, computes the optimal steering angle and rate required to align the vehicle's heading with the desired trajectory. This decision is then executed via electric power steering (EPS) or steer-by-wire systems, which eliminate the mechanical linkage between the steering wheel and the wheels, allowing for precise and programmable control.

Automated steering also adapts to various dynamic driving scenarios, including high-speed highway navigation, urban cornering, and obstacle avoidance. During obstacle-induced maneuvers, such as evasive swerving or lane shifting, the steering system must factor in the vehicle's lateral acceleration, yaw rate, and slip angle to avoid loss of traction and ensure maneuverability. In more advanced implementations, steering decisions are coordinated with longitudinal control systems (acceleration and braking) to enable seamless path following.

The evolution of automated steering systems is fundamental to achieving higher levels of driving automation, particularly from SAE Level 2 (partial automation) through Level 4 (high automation). The performance of such systems is continually enhanced through improvements in control theory, sensor integration, and real-time embedded processing.

2.4 MATLAB Simulink [6]

MATLAB Simulink is a powerful, block-diagram-based environment used extensively for the modeling, simulation, and analysis of dynamic systems. It is particularly suited for

representing systems that change over time—such as mechanical systems with moving parts, electrical circuits, control systems, and other real-time embedded applications. Developed by MathWorks, Simulink extends MATLAB's numerical computing environment by providing a visual interface where users can create models through interconnected blocks, each performing specific functions like mathematical operations, signal manipulation, logical decision-making, or time-based control. This graphical method not only enhances clarity in understanding system behavior but also accelerates the development cycle by eliminating the need for low-level coding during initial prototyping.

One of the most significant strengths of Simulink is its block-diagram modeling paradigm. In this approach, each element of the system—such as an input signal, mathematical operation, or output device—is represented as a block. These blocks are connected by lines that define how data flows between them. For instance, a PID controller block might receive an error signal, process it, and send an output to a motor block. This modular structure makes it easy to visualize, debug, and iterate over the design. It also encourages a hierarchical organization of systems, where subsystems can be encapsulated into blocks themselves, making large-scale system modeling both manageable and intuitive.

Simulink is used across a wide range of engineering disciplines, owing to its versatility and broad application base. In aerospace engineering, it is used to model flight dynamics, simulate control systems for aircraft, and develop autopilot algorithms. Aerospace companies often use Simulink in conjunction with Simscape and Aerospace Blockset to simulate physical phenomena like thrust, drag, altitude control, and orbital mechanics. In the automotive industry, Simulink plays a crucial role in developing systems like Anti-lock Braking Systems (ABS), Electric Power Steering (EPS), and vehicle stability controls. Its ability to interface with vehicle dynamics models allows engineers to validate their designs in a simulated environment before deploying them in real-time systems.

In robotics, Simulink is extensively used for developing and simulating robotic control systems, sensor data acquisition, trajectory planning, and feedback loops. Tools like Robotics System Toolbox and Simulink 3D Animation enhance its capability to model real-world robotic environments. Complex behaviors like simultaneous localization and mapping

(SLAM), inverse kinematics, and obstacle avoidance can be modeled and visualized. In biomedical engineering, Simulink is applied in the simulation of physiological systems such as cardiovascular flow dynamics or neural stimulation, and in developing wearable or implantable medical devices. These applications often rely on real-time data acquisition and system responses—capabilities that Simulink naturally supports.

One of Simulink's standout features is its integration with MATLAB scripting, allowing for an extended, programmable control of the simulation environment. Users can define parameters, generate random test data, perform batch simulations, and analyze the output using MATLAB scripts. This integration is crucial for researchers who wish to explore design spaces, tune parameters automatically, or validate large datasets generated through simulation.

Another critical aspect of Simulink is its support for real-time simulation and code generation. Engineers can use tools like Simulink Coder and Embedded Coder to convert models into C/C++ code that can be deployed onto embedded hardware. This is particularly useful for control systems or IoT applications, where real-time responsiveness and low-level control are required. Simulink also supports Hardware-in-the-Loop (HIL) testing, where simulated models are run in real-time alongside physical hardware components. This allows testing of embedded code under realistic conditions, significantly reducing development costs and risks associated with purely physical prototyping.

Simulink has been the foundation of countless academic and industrial projects aimed at designing and testing control systems. In automotive applications, for instance, it is commonly used to design and simulate Adaptive Cruise Control (ACC) systems that automatically adjust a vehicle's speed based on traffic conditions. Lane Keeping Assist (LKA) systems can be modeled using camera inputs simulated in Simulink and processed using logic blocks that control steering actuators. Vehicle platooning, a concept where multiple vehicles travel closely and in coordination using V2V communication, can also be visualized and tuned using Simulink's capabilities.

In robotics projects, Simulink has been used to simulate autonomous navigation algorithms that combine data from virtual sensors like LiDAR and ultrasonic range finders. Robot

Operating System (ROS) integration further extends Simulink's capabilities in real-world robotic applications. Using Simulink, researchers can experiment with virtual robots navigating through mazes or open fields, detecting obstacles, and planning safe trajectories without requiring physical hardware. This allows for extensive testing and refinement of control algorithms in a risk-free environment.

2.5 Machine Learning with Python [7]

Python has emerged as the most popular programming language for scientific computing, data analysis, and machine learning due to its simplicity, readability, extensive library ecosystem, and strong community support. It is an open-source, high-level language that supports multiple programming paradigms including object-oriented, imperative, and functional programming. Its syntax is particularly conducive to rapid prototyping, making it ideal for academic research and industrial applications in artificial intelligence (AI) and machine learning (ML). Python's versatility, coupled with powerful libraries, enables researchers and developers to implement and test complex machine learning algorithms with minimal effort.

In the context of machine learning, Python's strength lies in its mature ecosystem of libraries such as NumPy for numerical operations, Pandas for data manipulation and analysis, Matplotlib and Seaborn for visualization, and, most importantly, Scikit-learn, which provides simple and efficient tools for data mining and predictive modeling. These libraries offer a unified interface for classification, regression, clustering, dimensionality reduction, and model evaluation. Additionally, Python integrates well with Jupyter Notebooks, allowing researchers to combine live code, visualizations, and explanatory text in a single document—ideal for interactive experimentation and debugging.

2.5.1 Supervised Learning Algorithms

Supervised learning, the cornerstone of many AI applications, refers to the process of training a model using labeled data. In this paradigm, the algorithm learns a mapping from inputs to

outputs based on a training dataset. Python's Scikit-learn library offers several supervised learning algorithms that are widely used in both academia and industry:

- Decision Trees: A decision tree is a flowchart-like model that splits the dataset into subsets based on the value of input features. It is intuitive and interpretable, making it suitable for understanding the decision logic. Scikit-learn's Decision Tree Classifier supports various splitting criteria such as Gini impurity and entropy, enabling flexibility in model training.

- Random Forests: An ensemble method that combines multiple decision trees to improve prediction accuracy and reduce overfitting. Each tree is trained on a random subset of the data and features, and their predictions are aggregated. The Random Forest Classifier in Scikit-learn is known for its robustness and ease of tuning.

- K-Nearest Neighbors (KNN): KNN is a non-parametric method used for classification and regression. It classifies a new sample based on the majority label among its k nearest neighbors in the feature space. Although simple, KNN can perform well on well-distributed datasets and is often used as a baseline model.

These models are trained using preprocessed data, usually normalized or standardized, and their performance is evaluated using metrics such as accuracy, precision, recall, F1 score, and ROC-AUC. Cross-validation and grid search techniques are commonly employed to fine-tune hyperparameters and avoid overfitting.

2.5.2 Unsupervised Learning

Unsupervised Learning involves training on unlabeled data to discover hidden structures. Common algorithms include K-Means clustering and Principal Component Analysis (PCA), which are often used in exploratory data analysis and dimensionality reduction.

2.5.3 Reinforcement Learning

Reinforcement Learning (RL) focuses on training agents to make sequences of decisions by maximizing cumulative reward. Although not implemented in this project, RL techniques are commonly applied in autonomous systems, robotics, and game-playing agents.

2.6 Review of Previously Published Work

1) Fasoli Mauri – "Obstacle Avoidance Strategies for Omnidirectional Mobile Platforms"

This thesis [8] explores control strategies and navigation techniques for mobile omnidirectional robots, focusing on PID-based and intermediate-level path planning algorithms. The research develops a complete system for obstacle avoidance and navigation using sensors such as laser scanners. The trajectory planning is executed in real-time and the robot avoids dynamic and static obstacles using intermediate path planning techniques that balance global and local constraints. The thesis is implemented in MATLAB Simulink for simulation and testing. A suitable visual would be the trajectory planner showing planned vs. real path curves under obstacle constraints.

2) Giancarlo Pacitti – "Automated Driving System Design and Simulation Using MATLAB and Simulink"

This whitepaper [9] provides a structured approach to designing autonomous driving systems using MathWorks tools. It illustrates how engineers can simulate driving conditions and test sensor perception systems using virtual environments. The author explains how to design scenes and sensor fusion algorithms to validate logic for lane-following, vehicle detection, and path planning. The integration of the Unreal Engine for realistic scene rendering and the use of synthetic sensors for LiDAR, radar, and camera-based systems are emphasized. This simulation framework enables closed-loop testing of perception and control systems before deploying to physical vehicles.

3) Asif Ahmed – “Obstacle Avoidance System for Autonomous Driving”

This thesis [10] details the development of an obstacle avoidance subsystem as part of a model-based design approach for autonomous vehicles. The project follows the simplified V-model of software development, integrating requirements analysis, design, verification, and testing using Simulink. The model includes a behavior selector, trajectory planner, and vehicle controller. It emphasizes software-in-the-loop (SIL) testing and formal verification using tools like Simulink Design Verifier. The goal is to model a safety-critical subsystem capable of path planning and autonomous maneuvering around obstacles. An appropriate visual would be the Simulink block diagram showing the interactions among subsystems.

4) Fernando Castaño et al. – “Obstacle Recognition Based on Machine Learning for On-Chip LiDAR Sensors in a Cyber-Physical System”

This paper [11] presents an obstacle recognition library designed for a cyber-physical traffic system using on-chip LiDAR sensors. Integrated with SCANeR and MATLAB/Simulink, the system evaluates three AI-based recognition models—Multi-layer Perceptron (MLP), Self-Organizing Map (SOM), and Support Vector Machine (SVM). Results reveal that MLP performs best in sunny and foggy conditions, SVM in rainy environments, and SOM in snowy scenarios. This study demonstrates how different weather impacts obstacle detection accuracy. A valuable visual addition would be a comparison chart showing accuracy under different conditions for the three ML models.

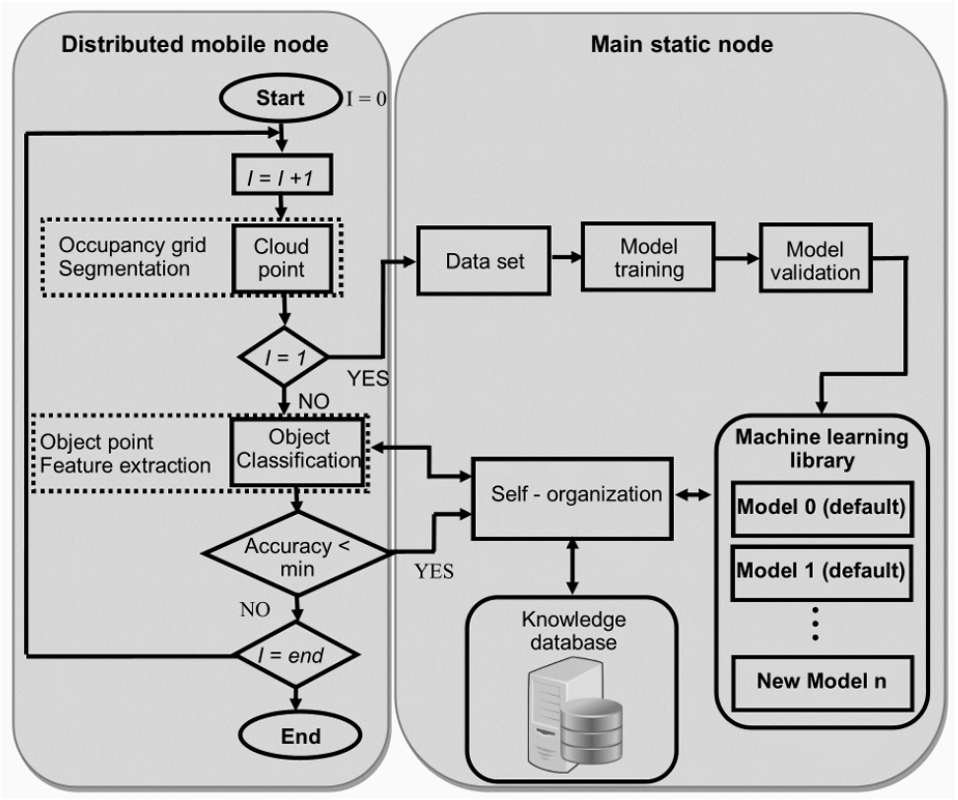


Figure 3. Block diagram of classification methodology for object-type identification

2.7 Summary of Literature Review

Autonomous driving systems rely on advanced technologies integrating perception, localization, planning, and control to navigate without human intervention. A critical component of such systems is obstacle detection, which uses multi-modal sensors—such as LiDAR, radar, and vision cameras—to identify and classify static and dynamic hazards. Sensor fusion algorithms, often implemented using machine learning frameworks, synthesize this sensory data to generate real-time environmental models for path planning and collision avoidance.

Modeling and simulation of such systems are extensively carried out using MATLAB Simulink, a graphical platform for dynamic system design. Simulink provides modular

block-diagram representations that help visualize control architectures such as lane keeping, adaptive cruise control, and obstacle avoidance. Through its integration with MATLAB scripting, users can fine-tune parameters, perform batch simulations, and analyze results programmatically. Real-time code generation using Simulink Coder and Embedded Coder enables deployment of simulation models onto embedded platforms, supporting Hardware-in-the-Loop (HIL) testing and verification. It also supports toolboxes for autonomous driving, robotics, and sensor modeling, making it ideal for developing perception-control loops.

Python plays a crucial role in implementing machine learning models for obstacle recognition and trajectory prediction. Its ecosystem includes libraries like Scikit-learn, NumPy, and Pandas for training supervised learning algorithms such as Decision Trees, Random Forests, and K-Nearest Neighbors. These are used to classify environmental features and detect potential obstacles. Python's compatibility with Jupyter Notebooks enhances iterative development and visualization.

Previous literature validates the use of MATLAB, Simulink, and Python for autonomous systems. Research studies demonstrate their use in model-based design, sensor fusion testing, and AI-driven perception systems under varied weather and environmental conditions. Collectively, these tools accelerate the prototyping, simulation, and deployment of safe, efficient autonomous driving solutions.

CHAPTER 3: MODEL DEVELOPMENT

3.1 Simulink Model

The Simulink model developed for this project represents the virtual simulation environment for evaluating the logic of automated obstacle detection and steering. This model captures the car's forward motion, obstacle placement, real-time collision detection, and steering response using a combination of Simulink blocks. The model also provides visual outputs for understanding and validating the performance of the system. Each component of the model is explained in the following subsections.

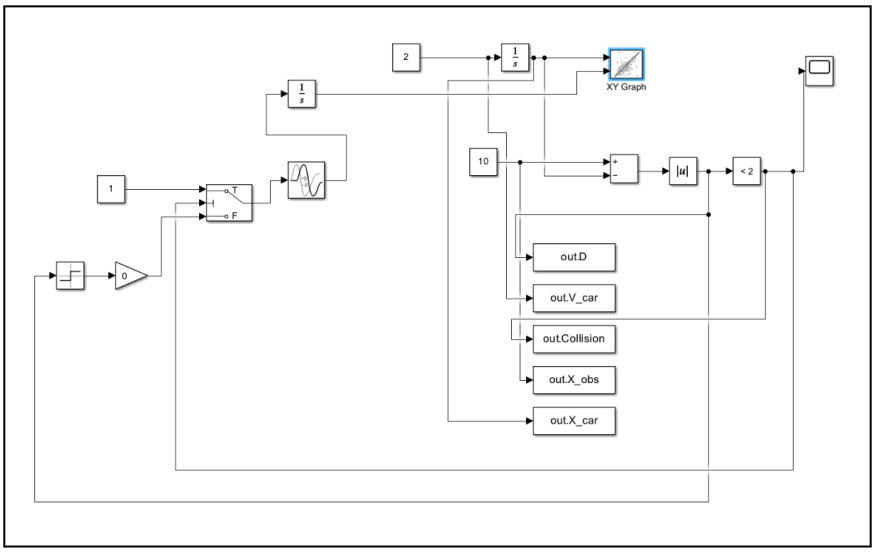


Figure 4. Simulink Model.

3.1.1 Vehicle Motion in X-Direction

The car's longitudinal position throughout the simulation. This forms the basis for tracking the car's motion relative to any nearby obstacles.The forward motion of the car is implemented along the X-axis using a basic kinematic relationship. A constant velocity input is provided using a Constant block with a value of 2, which means the vehicle moves at a

uniform speed of 2 units per second. This velocity is integrated over time using the Integrator block (represented by 1/s), resulting in the real-time position of the car along the X-axis. The output of this integrator block is labeled as out.X_car and represents

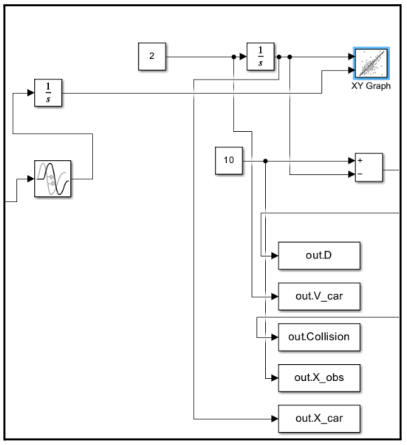


Figure 5. Vehicle Motion in X-Direction

3.1.2 Obstacle Placement and Distance Measurement

The obstacle in the simulation is positioned at a fixed location along the X-axis. This is done by using a Constant block with a set value, typically 10, representing the X-position of the obstacle. To compute how far the car is from the obstacle at any moment, a Subtraction block is used to calculate the difference between the obstacle's position and the car's current X-position. This value is then passed through an Absolute block to ensure the distance remains non-negative regardless of direction. The resulting signal is labeled as out.D and represents the dynamic distance between the car and the obstacle. This distance is a critical parameter for determining whether the car needs to take evasive action.

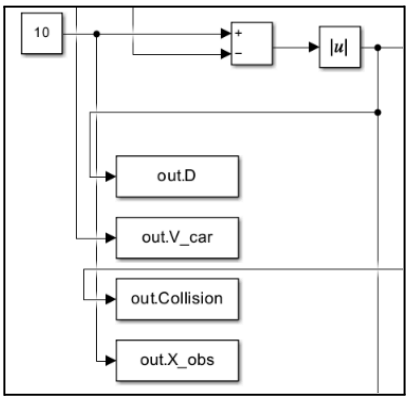


Figure 6. Obstacle Placement and Distance Measurement

3.1.3 Collision Detection Logic

Collision detection is handled using a conditional logic block. The output distance is passed into a Relational Operator block that checks whether the distance is less than 2 units. If this condition is met, it implies that the car is approaching too close to the obstacle, signaling a risk of collision. In such a case, the block outputs a Boolean value of 1 (true), otherwise it outputs 0 (false). This Boolean signal is recorded as out.Collision. The presence of this signal allows the system to distinguish between safe and unsafe conditions and plays a key role in triggering the vehicle's steering mechanism to avoid a crash.

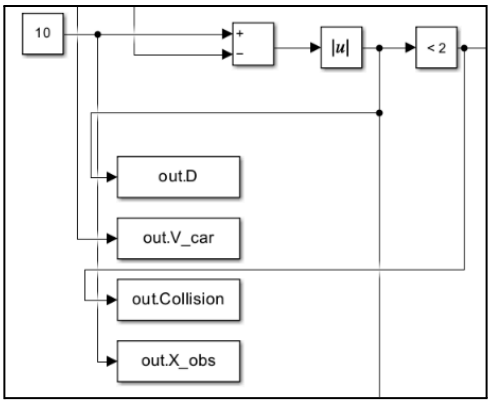


Figure 7. Collision Detection Logic

3.1.4 Steering Logic and Lane Change Behavior

The steering logic activates when a potential collision is detected. Under normal conditions, the vehicle continues to move in a straight line with no deviation in the Y-direction. However, when the collision detection signal is active, the system uses a Sine Wave block to simulate lateral deviation, representing a smooth lane change or steering maneuver. A Switch block governs whether the output follows the straight path or the sine wave motion based on the collision flag. If no obstacle is detected, the vehicle continues in a straight path; if an obstacle is near, the vehicle begins a diagonal or oscillating shift to avoid the object. This approach ensures that the vehicle's steering response is not abrupt, maintaining smoothness during lane changes.

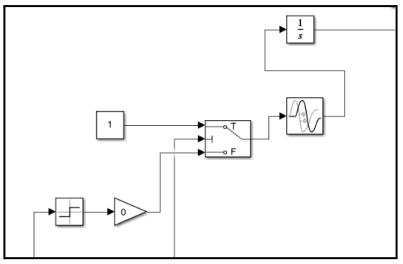


Figure 8. Steering Logic and Lane Change Behavior

3.1.5 Output Parameters

The model provides several output signals to monitor the behavior of the system. These include out.X_car, which gives the real-time X-position of the car; out.V_car, which represents the constant velocity input; out.x_obs, which shows the fixed position of the obstacle; out.D, the calculated distance between the car and obstacle; and out. Collision, the Boolean output indicating the presence or absence of a potential crash. These outputs are not only important for simulation visualization but also for data analysis and integration with external Python or machine learning-based modules.

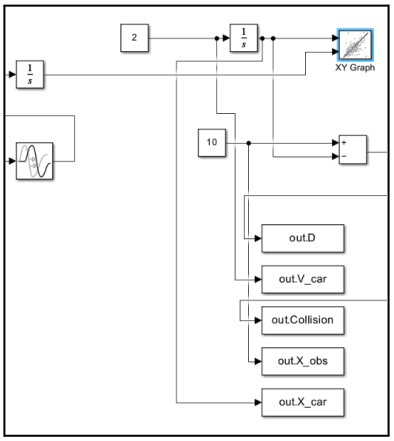


Figure 9. Output Parameters

3.1.6 Visualization with XY Graph

To visually observe the performance of the vehicle in relation to the obstacle, an XY block is used. This graph plots the X and Y positions of the car over time, showing how the car moves forward and whether it diverts from its straight path when an obstacle is detected. The trajectory change is clearly visible when the steering logic is triggered, allowing easy validation of the avoidance behavior. This visualization is a crucial component of the simulation, as it helps in understanding the efficiency and timing of the obstacle detection and avoidance logic implemented in the model.

3.2 Data Collection using MATLAB Code

```
obstacle_positions = [5, 7, 8, 9, 9.5, 9.7, 9.9, 10, 10.2, 10.5, 11, 12, 15, 18];
car_velocities = [1, 2, 3, 4];

data = []; % Initialize an empty matrix to store results

for i = 1:length(obstacle_positions)
    for j = 1:length(car_velocities)
        % Set obstacle position and car velocity
        X_obs_value = obstacle_positions(i);
        V_car_value = car_velocities(j);

        % Assign values in Simulink
        assignin('base', 'X_obs_value', X_obs_value);
        assignin('base', 'V_car_value', V_car_value);

        % Run the simulation
        simOut = sim('Obstacle_avoidance_bot');
        % Extract results
        X_car = simOut.X_car.signals.values;
        X_obs = simOut.X_obs.signals.values;
        D = simOut.D.signals.values;
        V_car = simOut.V_car.signals.values;
        Collision = simOut.Collision.signals.values;

        num_samples = length(X_car);

        % Repeat single values to match time-series length
        X_obs = repmat(X_obs, num_samples, 1);
        V_car = repmat(V_car, num_samples, 1);

        % Append new data
        data = [data; X_car, X_obs, D, V_car, Collision];
    end
end

% Convert data to table and save as CSV
data_table = array2table(data, 'VariableNames', {'X_car', 'X_obs', 'Distance',↙
'Velocity', 'Collision'});
writetable(data_table, 'obstacle_data.csv');

disp(" Data collection complete. CSV file saved!");
```

Figure 10. Data Collection Using MATLAB

The success of any machine learning-based obstacle detection system largely depends on the quality and relevance of the training dataset. In this project, a robust dataset was generated using MATLAB and Simulink to support the development of a predictive model that could identify potential collisions in real time. The simulation environment was built using Simulink and was capable of modeling dynamic interactions between an autonomous vehicle and multiple road obstacles. The data collection phase was designed to systematically cover a

wide range of obstacle positions and vehicle velocities, simulating realistic driving conditions. These simulations were not only safe and repeatable but also offered full control over parameters, enabling the generation of high-quality time-series data for supervised learning.

The primary objective during this stage was to create a labeled dataset containing all relevant features that could be used to predict the likelihood of a collision. Key variables such as the longitudinal position of the car (X_car), the obstacle's position (X_obs), the distance between the car and the obstacle, the car's velocity (Velocity), and the resulting collision status (Collision) were recorded. By altering obstacle configurations and vehicle dynamics across multiple simulation runs, the dataset captured both collision and non-collision scenarios, which are essential for training a classifier capable of generalization. The generated dataset was exported in the form of a CSV file named obstacle_data.csv, which was later imported into Python for model training and validation.

3.2.1 Automated Scenario Generation Using MATLAB and Simulink

To automate the dataset generation process, a MATLAB script was developed to control and execute multiple simulations through systematic parameter variation. Two primary input parameters were varied during this process: the obstacle's position and the vehicle's velocity. Obstacle positions were randomly selected within a fixed range from 5 to 18 meters along the road, while the vehicle velocity was varied from 1 to 4 meters per second. These parameters were chosen to reflect a realistic spread of driving situations that an autonomous system may encounter.

For every combination of obstacle position and velocity, the corresponding values were assigned into MATLAB's base workspace using the assignin function. These values were then passed as inputs into the Simulink model named Obstacle_avoidance_bot.slx. The sim() function was used to run the simulation for a fixed duration, during which the car's movement, obstacle detection logic, and collision flags were computed dynamically. The simulation output included signals for the vehicle's X position, the fixed obstacle location, the real-time calculated distance, and the binary collision indicator. Since the obstacle position

and velocity remained constant throughout each simulation, these were expanded using the repmat function to align with the time dimension of the other variables.

All output signals from a simulation run were organized into a matrix, converted into a structured table, and appended to a growing dataset inside the MATLAB workspace. Once all iterations across the full range of parameter combinations were completed, the complete dataset was exported as a CSV file using the writetable command. This CSV file served as the core dataset used in the machine learning phase of the project for training and evaluating predictive models such as Decision Trees, Random Forests, and K-Nearest Neighbors.

The use of Simulink for data generation provided several advantages. It enabled closed-loop simulation with full control over inputs, allowed accurate modeling of motion and detection logic, and made the process of reproducing specific scenarios highly efficient. Additionally, because the data was synthetically generated in a structured manner, it avoided common pitfalls of real-world datasets, such as sensor noise, missing data, or class imbalance. This made the dataset particularly well-suited for initial model development, allowing rapid prototyping of classifiers before deployment in more complex environments.

3.3 Python Code

```python
import pandas as pd
import numpy as np
import joblib
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Load dataset
data = pd.read_csv('obstacle_data.csv')

# Features and labels
X = data[['X_car', 'X_obs', 'Distance', 'Velocity', 'Y_car']].values
y = data['Collision'].values

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model
dt_model = DecisionTreeClassifier(max_depth=10, min_samples_split=5, random_state=42)
dt_model.fit(X_train, y_train)

# Evaluate and save
accuracy_dt = accuracy_score(y_test, dt_model.predict(X_test))
print(f"Decision Tree Model Accuracy: {accuracy_dt * 100:.2f}%")
joblib.dump(dt_model, "decision_tree_obstacle.pkl")
print("Model saved as decision_tree_obstacle.pkl")

# Plot decision tree
plt.figure(figsize=(10, 5))
plot_tree(dt_model, feature_names=['X_car', 'X_obs', 'Distance', 'Velocity', 'Y_car'],
          class_names=['No Collision', 'Collision'], filled=True, rounded=True)
plt.title("Decision Tree for Obstacle Prediction")
plt.show()

# Load model for prediction
model = joblib.load("decision_tree_obstacle.pkl")

# Prediction function
def predict_lane_change(obstacle_data):
    obstacle_data = np.array(obstacle_data, dtype=float).reshape(1, -1)
    prediction = model.predict(obstacle_data)[0]
    lane_change = np.random.choice([-1, 1]) if prediction == 1 else 0
    return int(prediction), lane_change
```

Figure 11. Model Training(Python) Using Decision Tree

3.3.1 Dataset Preparation and Feature Engineering

The obstacle prediction system for the autonomous vehicle simulation was developed using Python and integrated machine learning techniques. The primary goal was to determine whether a potential collision would occur between the ego vehicle and any surrounding obstacles, based on real-time kinematic data.

The dataset used for training was generated in MATLAB, as described earlier in Section 3.2, and exported as a CSV file named obstacle_data.csv. This dataset includes features such as the longitudinal position of the car (X_car), the position of the obstacle (X_obs), the relative distance between them, the velocity of the car, and its lateral position in the lane (Y_car). Each data sample was labeled with a binary value under the Collision column, indicating whether a collision was predicted (1) or not (0) during that MATLAB simulation step.

3.3.2 Model Selection and Training

To implement this prediction task, several libraries in Python were employed. Pandas and NumPy were used for data handling and numerical operations. Scikit-learn, a widely used machine learning toolkit, was used for model training, evaluation, and visualization.

Initially, multiple machine learning models were tested, including K-Nearest Neighbors (KNN), Random Forest, Support Vector Machine (SVM), and Logistic Regression. After rigorous experimentation and comparison, the Decision Tree Classifier emerged as the most suitable model due to its high prediction accuracy and interpretability. With a test accuracy of over 95%, it outperformed the other models in both consistency and performance.

The data was divided into training and testing subsets in an 80:20 ratio, ensuring that the model would be validated on unseen data for an unbiased evaluation. The Decision Tree Classifier was trained with a maximum depth of 10 and a minimum sample split of 5 to prevent overfitting and maintain generalization.

3.3.3 Model Serialization and MATLAB Interfacing

Once the model was trained and evaluated, it was serialized and saved in the form of a .pkl file using the Joblib library. A .pkl file, or pickle file, is a format used to store trained models so they can be loaded later without the need for retraining.

This feature becomes particularly important in this project, as the model needs to be re-used in MATLAB for the final GUI implementation. By importing the .pkl file directly into MATLAB using appropriate interfacing methods, the GUI can make real-time decisions on car motion based on obstacle predictions from the trained model.

3.3.4 Model Visualization and Explainability

In addition to training and saving the model, a decision tree plot was generated using Matplotlib to visualize the internal structure of the trained classifier. This graphical representation helps in understanding how the model uses different input features to make predictions about collisions. Such transparency is valuable for debugging, model optimization, and ensuring reliable behavior when deployed in autonomous driving environments.

3.3.5 Real-Time Prediction Function for Simulation Integration

Furthermore, a prediction function was defined in Python to allow real-time obstacle data to be passed into the model for inference. This function takes a new input sample, reshapes it appropriately, and uses the trained decision tree to predict the occurrence of a collision. Based on the output, it suggests a lane change. If a collision is predicted, the function randomly chooses a lane change direction, either -1 or +1, simulating the car’s attempt to avoid the obstacle. If no collision is predicted, the car maintains its current lane with a zero lane-change command.

This function will be directly utilized in the MATLAB GUI to demonstrate autonomous steering logic based on live obstacle detection. Overall, this Python-based module serves as a crucial component of the obstacle detection and avoidance pipeline. It not only leverages supervised learning techniques effectively but also enables seamless integration with the MATLAB-based simulation and GUI design. The ability to train, validate, visualize, and deploy the model across platforms highlights the collaborative strength of Python and MATLAB in building intelligent, cross-disciplinary engineering systems.

3.4 Python to MATLAB

```
% Set Python Environment
pyenv('Version', 'C:\Users\Grishma\AppData\Local\Programs\Python\Python312\python.exe');
script_path = "C:\Users\Grishma\Documents\MATLAB\obstaclepythonpyfile.py";

% Execute Python script
try
    command = "python """ + script_path + """";  % Run script to train model
    [status, cmdout] = system(command);
    if status ~= 0
        disp("Python script error:"); disp(cmdout); return;
    end
    py.importlib.import_module('obstaclepythonpyfile');  % Import for prediction
catch ME
    disp("Error executing Python script:"); disp(ME.message); return;
end

% Initialize figure
figure; hold on; grid on;
xlim([0 50]); ylim([0 3]);
xlabel('X Position'); ylabel('Track');
title('Obstacle Avoidance Simulation');

% Car and obstacle setup
X_car = 0; Y_car = 2; speed = 0.25;
num_obstacles = 13;
max_distance = 50;
obstacles = [randi([5, 50], num_obstacles, 1), randi([1, 3], num_obstacles, 1)];

while X_car < max_distance
    % Detect nearest obstacle in current lane
    lane_obstacles = obstacles(obstacles(:,2) == Y_car, :);
```

Figure 12. Python-MATLAB Integration - I

```
32
33          if isempty(lane_obstacles)
34              min_dist = Inf; X_obs = Inf; Y_obs = Y_car;
35          else
36              [min_dist, idx] = min(abs(lane_obstacles(:,1) - X_car));
37              X_obs = lane_obstacles(idx, 1);
38              Y_obs = lane_obstacles(idx, 2);
39          end
40
41          % Predict using Python
42          result = py.obstaclepythonpyfile.predict_lane_change([X_car, X_obs, min_dist, 2, Y_car]);
43          prediction = double(result{1}); lane_change = double(result{2});
44
45          % Lane change with diagonal motion
46          if prediction == 1 && min_dist <= 2
47              steps = 10;
48              lane_target = min(max(Y_car + lane_change, 1), 3);
49              dx = speed * 0.3;
50              dy = (lane_target - Y_car) / steps;
51
52              for i = 1:steps
53                  X_car = X_car + dx;
54                  Y_car = Y_car + dy;
55
56                  % Move obstacles left as car moves forward
57                  obstacles(:,1) = obstacles(:,1) - dx;
58                  for j = 1:num_obstacles
59                      if obstacles(j,1) < X_car + 1
60                          obstacles(j,1) = X_car + randi([20, 30]);
61                          obstacles(j,2) = randi([1,3]);
62                      end
```

Figure 13. Python-MATLAB Integration - II

```
63              end
64
65              % === DRAW SCENE ===
66              clf;
67
68              % Grass on left and right
69              fill([X_car-5, X_car+15, X_car+15, X_car-5], [3.5, 3.5, 4.5, 4.5], [0.3 0.7 0.3]); % Top grass
70              fill([X_car-5, X_car+15, X_car+15, X_car-5], [-1, 0, 0, -1], [0.3 0.7 0.3]);        % Bottom grass
71
72              % Road
73              fill([X_car-5, X_car+15, X_car+15, X_car-5], [0, 0, 3.5, 3.5], [0.6 0.6 0.6]); % Grey road
74
75              hold on; grid on;
76
77              % Lane lines
78              for y_line = [0.5, 1.5, 2.5, 3.5]
79                  plot([X_car-5, X_car+15], [y_line, y_line], 'w--', 'LineWidth', 1.5);
80              end
81
82              xlim([X_car-5, X_car+15]); ylim([-0.5, 4.5]);
83
84              % Draw obstacles
85              for j = 1:size(obstacles,1)
86                  text(obstacles(j,1), obstacles(j,2), "🚧", 'FontSize', 20, 'HorizontalAlignment', 'center');
87              end
88              % Draw car
89              text(X_car, Y_car, "🚗", 'FontSize', 20, 'HorizontalAlignment', 'center');
90              title("Changing Lane", 'FontSize', 14);
91
92              pause(0.05);
93          end
```

Figure 14. Python-MATLAB Integration - III

```
        Y_car = round(lane_target);
    else
        X_car = X_car + speed;

        % Move obstacles left as car moves forward
        obstacles(:,1) = obstacles(:,1) - speed;
        for j = 1:num_obstacles
            if obstacles(j,1) < X_car + 1
                obstacles(j,1) = X_car + randi([20, 30]);
                obstacles(j,2) = randi([1,3]);
            end
        end
    end

    % === DRAW SCENE ===
    clf;

    % Grass on both sides
    fill([X_car-5, X_car+15, X_car+15, X_car-5], [3.5, 3.5, 4.5, 4.5], [0.3 0.7 0.3]); % Top grass
    fill([X_car-5, X_car+15, X_car+15, X_car-5], [-1, 0, 0, -1], [0.3 0.7 0.3]);         % Bottom grass

    % Road
    fill([X_car-5, X_car+15, X_car+15, X_car-5], [0, 0, 3.5, 3.5], [0.6 0.6 0.6]);

    hold on; grid on;

    % Lane lines
    for y_line = [0.5, 1.5, 2.5, 3.5]
        plot([X_car-5, X_car+15], [y_line, y_line], 'w--', 'LineWidth', 1.5);
    end
```

Figure 15. Python-MATLAB Integration - IV

```
    xlim([X_car-5, X_car+15]); ylim([-0.5, 4.5]);

    % Draw obstacles
    for j = 1:size(obstacles,1)
        text(obstacles(j,1), obstacles(j,2), "🚧", 'FontSize', 20, 'HorizontalAlignment', 'center');
    end

    % Draw car
    text(X_car, Y_car, "🚗", 'FontSize', 20, 'HorizontalAlignment', 'center');

    % Status message
    if prediction == 1 && min_dist <= 2
        title("Lane Change Complete", 'FontSize', 14);
    else
        title("Safe", 'FontSize', 14);
    end

    pause(0.1);
end
```

Figure 16. Python-MATLAB Integration - V

3.4.1 Integration of Python with MATLAB

The final integration of the obstacle detection and avoidance system was implemented in MATLAB, where the graphical user interface (GUI) simulation was designed to visually demonstrate the autonomous vehicle's decision-making behavior in the presence of obstacles. This portion of the project served as the culmination of earlier Python-based machine learning

model development and MATLAB-based simulation data generation. The code links MATLAB's high-quality visualization and control capabilities with Python's computational machine learning backend, effectively combining the strengths of both platforms.

The simulation begins by configuring the MATLAB environment to recognize and communicate with the Python interpreter. Using the pyenv function, the exact location of the Python 3.12 executable on the local system is specified to ensure that MATLAB can execute Python commands correctly. Subsequently, a Python script containing the model training and prediction logic—saved as obstaclepythonpyfile.py—is executed directly from MATLAB using the system command. This command ensures that the latest version of the Decision Tree Classifier is trained and saved into a .pkl file if needed. To enable runtime prediction capability, the Python module is imported using py.importlib.import_module, making its prediction function callable from within MATLAB.

3.4.2 Simulation Design and Obstacle Management

Once the Python environment is set up, the simulation initializes a 2D graphical scene to represent the car and obstacles on a three-lane road. A figure window is created with defined X and Y axes to simulate a track view from above. The car's initial position is set to the beginning of the road at lane 2 (Y_car = 2), and a constant forward velocity (speed = 0.25) is defined. Thirteen obstacles are randomly generated and placed along the road in one of the three available lanes, with randomized longitudinal positions. These obstacles are meant to simulate construction zones, stationary vehicles, or other roadblocks that the car might encounter.

The main simulation loop continues as long as the car's position (X_car) is within the defined length of the road (max_distance = 50). In each iteration, the car searches for obstacles present in its current lane. If any are detected, it calculates the distance to the nearest one by comparing longitudinal coordinates. If no obstacle is found, it assumes infinite distance, allowing the car to continue without concern.

At every simulation step, the script calls the Python prediction function by passing the current values of the car's position, the obstacle's position, the calculated distance, vehicle speed, and the current lane. The Python function, using the trained Decision Tree Classifier, returns a binary collision prediction and a suggested lane change direction. This output is extracted using Python-to-MATLAB type conversion and used to inform the next course of action in the simulation.

3.4.3 Lane Changing Logic and Animation

If a collision is predicted and the detected obstacle is within a threshold distance of 2 units, a lane change is initiated. To simulate a more natural maneuver, this transition is not instantaneous but occurs gradually over ten steps. The target lane is calculated based on the predicted direction, ensuring it remains within the valid bounds of the road (lanes 1 to 3). During each of these ten steps, the car moves slightly forward (dx) and also shifts laterally (dy), creating a diagonal motion effect that mimics a real-world lane change.

Simultaneously, all obstacles move leftward relative to the car's progress, giving the illusion of forward motion. If any obstacle moves too far behind the car, it is repositioned ahead with new random coordinates to maintain obstacle density throughout the simulation.

Throughout the process, the graphical scene is updated using MATLAB's plotting and annotation capabilities. Green rectangles on the top and bottom edges represent grass areas, while the central grey strip signifies the road. Dashed white lines mark the lane divisions. Obstacles are drawn as roadblocks ("🚧"), and the car is displayed as a vehicle emoji ("🚗"). Titles such as "Changing Lane", "Safe", and "Lane Change Complete" dynamically reflect the car's decision at each step, improving interpretability for users.

If no collision is predicted, the car continues moving forward without lateral deviation. The obstacle positions are updated similarly, and the visual scene is refreshed to maintain

real-time responsiveness. The use of pause commands ensures smooth and observable animation without overwhelming system performance.

This MATLAB code effectively demonstrates how an autonomous system can use real-time data and machine learning predictions to make intelligent navigation decisions. The ability to call Python scripts and prediction functions within MATLAB not only ensures seamless cross-platform integration but also exemplifies how simulation tools and computational environments can work hand-in-hand. The .pkl file generated by Python is especially critical here, as it stores the trained machine learning model, allowing MATLAB to leverage complex learned behaviors without needing to implement or retrain the model from scratch.

This interoperability is vital for applications such as advanced driver-assistance systems (ADAS), where modularity, efficiency, and real-time responsiveness are essential. This integrated MATLAB-Python simulation offers a visually engaging and technically sound platform to model autonomous vehicle behavior in the presence of road obstacles. By combining data-driven predictions with visual feedback, the system demonstrates how computational intelligence can be deployed for real-world navigation challenges.

CHAPTER 4: RESULTS AND DISCUSSIONS

4.1 Obstacle Detecting Simulator

The obstacle detection simulator developed in this project was a critical component of the overall autonomous vehicle framework. Built using MATLAB Simulink and discussed in detail in Section 3.1, the model was designed to simulate real-time obstacle detection and steering response. The goal of this simulator was to mimic the working of a LiDAR sensor that scans the vehicle's surroundings and provides critical information regarding the distance between the vehicle and nearby obstacles. Based on the output of this simulation, the car either continues its straight-line motion or initiates a steering correction to change lanes and avoid a potential collision.

The simulator takes into account both the longitudinal motion of the vehicle (in the X-direction) and the location of a fixed obstacle placed on its path. As the simulation progresses, the system continuously calculates the distance between the moving car and the static obstacle. A comparator block checks whether this distance drops below a threshold value (set to 2 units). If this condition is satisfied, the model flags the situation as a potential collision, thus triggering an avoidance response. Two key outputs were generated from this simulator, both of which are examined in this section. These outputs were visualized through a Scope block and an XY Graph block, representing discrete and continuous behavior respectively.

4.1.1 Output 1 – Collision Detection Scope Signal



Figure 17. Scope Result of Simulink Model

The first output of the simulator corresponds to the binary decision signal that determines whether a collision is imminent. This output is visualized using a Scope block. The Scope captures the boolean output signal from the Collision Detection logic, which is derived using a comparison block that checks whether the calculated distance D between the car and the obstacle is less than the specified threshold of 2 units.

As seen in the figure 17, the signal starts with a value of 0, indicating a safe zone where the car is far from the obstacle. As time progresses and the car moves forward, the distance gradually decreases. At approximately t = 7 seconds, the car enters the danger zone (distance < 2), and the boolean flag switches from 0 to 1. This spike in the signal represents the simulator's detection of a potential collision. The signal then returns to 0 after the car moves past the obstacle or changes its lane, depending on the steering logic enabled.

This binary spike acts as a virtual replication of what a LiDAR sensor might send to the vehicle’s control system—a simple high or low signal based on obstacle proximity. The clean

transition from 0 to 1 in the Scope plot validates that the model is successfully detecting potential collision conditions based on position data. Furthermore, it also serves as a trigger for activating the Y-direction steering mechanism that shifts the car to a new lane. This kind of decision-making signal is vital in real-world autonomous systems, where a split-second decision can prevent a crash.

The Scope output confirms that the collision detection logic in the Simulink model functions as intended. It accurately identifies the threshold-crossing event, replicates a LiDAR-type proximity detection scenario, and signals the vehicle controller to initiate evasive action. This output serves as the foundational proof-of-concept for the car's perception and response logic, laying the groundwork for real-time control implementation and machine learning integration.

4.1.2 Output 2 – XY Graph Trajectory Visualization

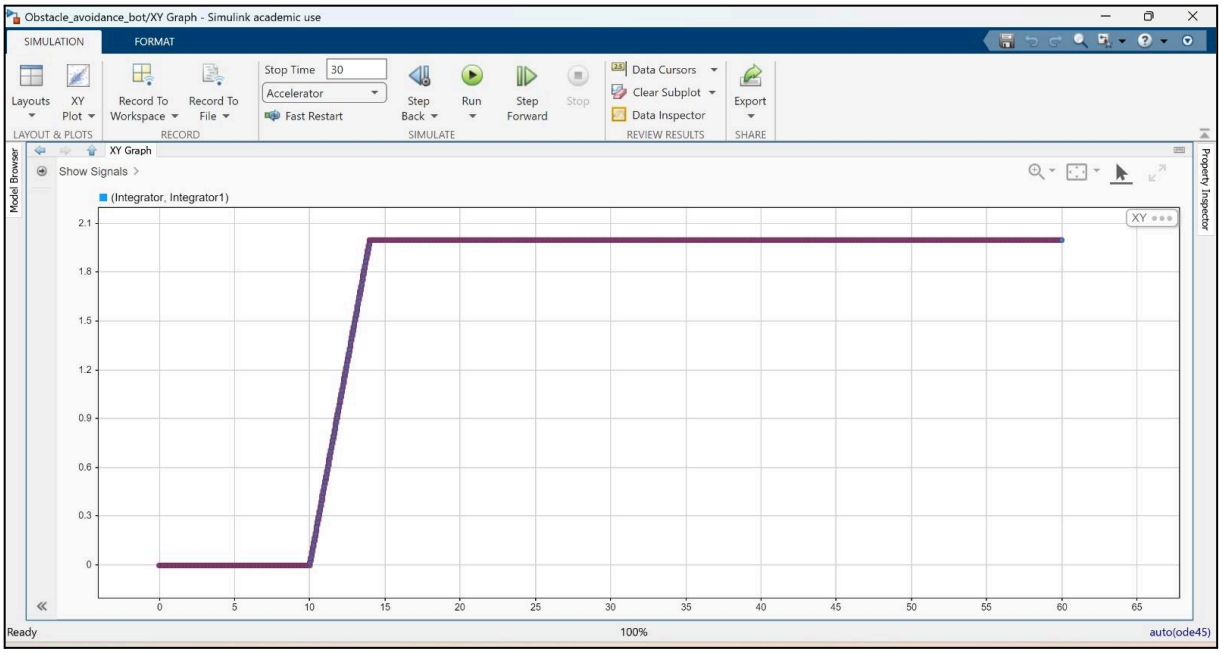


Figure 18. XY Plot Result of Simulink Model

The second output of the obstacle detection simulator is the continuous visual representation of the car's motion in two dimensions—X and Y—plotted using an XY Graph block. This visualization, shown in figure 18, provides a real-time trajectory plot of the vehicle as it

moves forward and responds to obstacle detection. While the Scope plot previously discussed focuses on the binary collision status, this output provides a more intuitive and human-readable representation of the car's physical behavior under various simulation conditions.

The XY Graph plots the car's X-position (forward movement) against its Y-position (lane indicator). At the start of the simulation, the car moves steadily in a straight line, reflecting motion strictly in the X-direction. The Y-coordinate remains constant, indicating that the car is maintaining its lane and no obstacle is presently in the detection zone. This behavior is seen in the XY plot as a perfectly horizontal line.

However, once the simulated LiDAR logic detects a nearby obstacle (i.e., when the calculated distance falls below 2 units), the collision flag is activated, and the model transitions into an evasive steering response. Crucially, this steering behavior is not executed abruptly. Instead of instantly shifting the car to another lane, which would be unrealistic and physically infeasible, the model uses a more natural transition by incrementally adjusting the car's Y-position over a series of small steps.

This is achieved through the use of an auxiliary Y-velocity component in the steering logic. Once a collision is detected, a diagonal motion is initiated by dividing the lane change into small time-based increments. For a fixed number of steps (typically 10), a small Y-offset is added in each iteration, while the X-position continues to increase. This results in a smooth diagonal curve on the XY Graph—representing the car gently drifting into the adjacent lane, much like how a real vehicle would steer in response to an obstacle. During this maneuver, the vehicle's forward velocity is also slightly reduced (e.g., by 30%) to account for safe lane transition, mimicking real-world driving dynamics.

This gradual change eliminates the visual and behavioral discontinuities that would occur with an instantaneous Y-jump, enhancing both realism and safety. The result is a smooth, continuous arc visible on the XY Graph. After completing the lane shift, the car stabilizes in the new lane and resumes straight-line motion, reflected on the graph as another horizontal line at a different Y-level.

This trajectory output serves as a critical validation tool. It confirms that the car not only detects obstacles but also responds in a realistic, controlled manner. The XY Graph clearly visualizes each segment of the response—the detection point, the transition path, and the new trajectory—thereby verifying both the perception and actuation logic. It is also a testament to the model's attention to physical feasibility and smooth control, demonstrating how simulation can be leveraged to test real-world behaviors before hardware implementation.

The XY Graph not only illustrates the car's ability to avoid obstacles but also showcases the success of the implemented smooth steering algorithm. This steering transition, powered by incremental Y-motion and reduced X-velocity during evasive action, reinforces the accuracy and naturalness of the avoidance behavior in the simulated autonomous system.

4.2 Data as Comma Separated Values

	A	B	C	D	E	F
1	X_car	X_obs	Distance	Velocity	Y_car	Collision
18442	7.786229	10	2.213771	2	2	0
18443	7.806229	10	2.193771	2	2	0
18444	7.826229	10	2.173771	2	2	0
18445	7.846229	10	2.153771	2	2	0
18446	7.866229	10	2.133771	2	2	0
18447	7.886229	10	2.113771	2	2	0
18448	7.906229	10	2.093771	2	2	0
18449	7.926229	10	2.073771	2	2	0
18450	7.946229	10	2.053771	2	2	0
18451	7.966229	10	2.033771	2	2	0
18452	7.986229	10	2.013771	2	2	0
18453	8	10	2	2	2	0
18454	8	10	2	2	2	0
18455	8	10	2	2	2	1
18456	8.02	10	1.98	2	2	1
18457	8.04	10	1.96	2	2	1
18458	8.06	10	1.94	2	2	1
18459	8.08	10	1.92	2	2	1
18460	8.1	10	1.9	2	2	1

obstacle_data

Figure 19. Comma Separated Values File Snippet

In addition to real-time simulation and visualization, an important goal of this project was to generate a structured dataset that could be used for offline analysis and machine learning

model training. To achieve this, a MATLAB script was implemented to run the Simulink model titled "Obstacle_avoidance_bot" multiple times using varying obstacle positions and car velocities. This code systematically extracted simulation outputs and saved them in a CSV (Comma-Separated Values) file format, which is widely used for data storage and interoperability between MATLAB and Python environments.

The MATLAB code first initializes two vectors: obstacle_positions and car_velocities. These vectors contain a range of values to represent different obstacle distances (from 5 to 18 units) and various speeds at which the car may travel (1 to 4 units). These parameters serve as input conditions for the simulator. A nested loop structure is then used to iterate through every possible combination of obstacle position and car velocity. For each such combination, the script assigns the respective values to the MATLAB base workspace using the assignin function. These values are automatically picked up by the Simulink model at runtime as they are connected to Constant blocks or similar parameter blocks within the model (defined in Section 3.2).

Once the parameters are assigned, the Simulink model is executed using the sim command. The output of each simulation run is stored in the simOut object, which contains time-series signal data for various outputs. These include the car's position (X_car), the obstacle's fixed position (X_obs), the distance between them (D), the velocity of the car (V_car), and the collision status (Collision). The variable Y_car, though not explicitly extracted in the script, is assumed to be constant for this dataset and can be appended if required later.

Because some of these outputs are scalar values (such as X_obs and V_car) while others are time-varying (like X_car and D), the scalar values are repeated to match the length of the time-series data using MATLAB's repmat function. This ensures that each row of the resulting dataset corresponds to a complete observation of the simulation at a single time step, making it suitable for machine learning and time-series analysis.

All the processed data from each iteration is concatenated into a matrix called data, which grows with each loop execution. After all iterations are completed, the full dataset is converted into a table using the array2table function, with appropriate variable names

assigned to each column: 'X_car', 'X_obs', 'Distance', 'Velocity', and 'Collision'. The table is then exported as a CSV file named obstacle_data.csv using the writetable function. Upon successful completion, the terminal displays a confirmation message: “Data collection complete. CSV file saved!”

The resulting CSV file contains a total of 168,449 data rows, with each row representing one timestamp of simulated behavior under a specific obstacle and velocity configuration. The dataset is rich in variability, capturing both collision and non-collision scenarios. This comprehensive dataset serves as the backbone for training machine learning models (explained in later sections), enabling classification of collision risk based on dynamic vehicle and obstacle parameters. Moreover, the CSV format ensures compatibility with Python, Excel, and other data analysis tools, making it a valuable resource for further research and algorithm testing.

4.3 Decision Tree

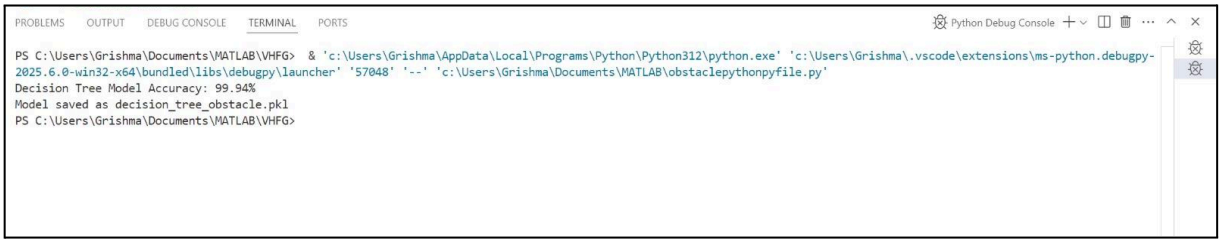


Figure 20. Python Code Output Terminal

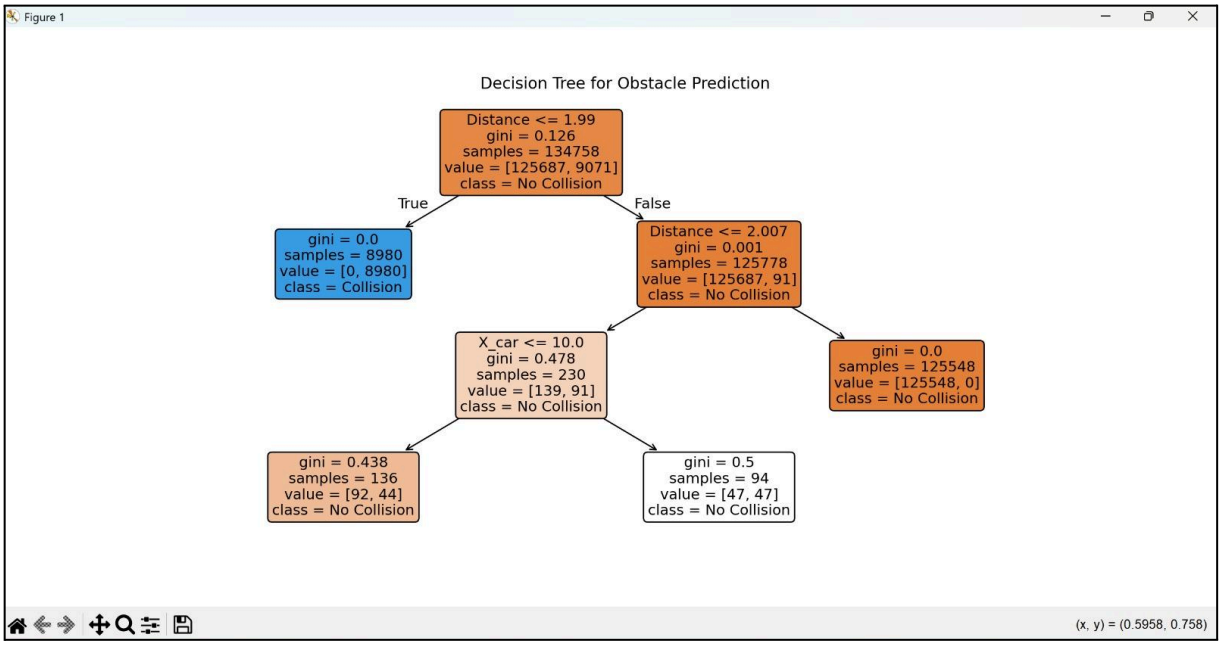


Figure 21. Decision Tree

In the context of autonomous vehicle systems, real-time decision-making based on environmental cues is of utmost importance. One of the primary goals of this project was to enable the simulated car to intelligently detect and react to potential collisions using data generated from the obstacle avoidance simulator. Section 4.3 discusses the implementation, training, evaluation, and interpretation of a Decision Tree Classifier developed in Python Code of section 3.3 using the Scikit-learn library. This classifier was trained on the dataset generated and saved in CSV format as explained in Section 4.2.

4.3.1 Dataset and Feature Selection

The dataset used for training the model consisted of over 168,000 samples, each representing a timestamp in the simulation. For each sample, the following features were extracted: X_car (car's longitudinal position), X_obs (obstacle position), Distance (separation between car and obstacle), Velocity (speed of the car), and Y_car (lane number). The target label for classification was Collision, a binary indicator representing whether a collision occurred at that timestamp (1 for collision, 0 for no collision).

The dataset was first read into a Pandas DataFrame and then split into training and testing sets using an 80-20 ratio with a fixed random seed for reproducibility. The resulting training set was used to fit a Decision Tree model, while the testing set was used to evaluate its performance.

4.3.2 Model Training and Accuracy

The model was built using the DecisionTreeClassifier from Scikit-learn. It was configured with a maximum depth of 10 to prevent overfitting and a minimum samples split of 5 to ensure that each node had sufficient training data. After training, the model was evaluated on the test set, yielding an impressive accuracy of 99.94%. This high accuracy demonstrates that the dataset contained clear and distinguishable patterns that the tree could learn effectively.

The trained model was serialized using Joblib and saved as "decision_tree_obstacle.pkl" for further use in simulation and real-time prediction. This persistent storage of the model allowed it to be loaded quickly during live execution and integrated seamlessly with the MATLAB-based steering simulator via Python-MATLAB interface.

4.3.3 Visualization and Interpretation of the Tree

The structure of the trained decision tree was visualized using the plot_tree function. The resulting diagram (as shown in the image) displays how the model makes decisions based on the feature thresholds. At the root of the tree is the most important feature: Distance. The first decision node checks whether Distance is less than or equal to 1.99. If true, the model moves to the left child node, where nearly 9,000 samples all belong to the "Collision" class, leading to a Gini impurity of 0.0—indicating perfect classification. This clearly validates the importance of proximity in predicting a collision, as expected.

If the Distance is greater than 1.99, the model proceeds to the right child node, where a second threshold checks if the Distance is less than or equal to 2.007. This tight decision boundary (just 0.017 units wide) reflects how finely the model has tuned its split to distinguish borderline cases between safe and unsafe distances. Most of the samples on this side of the tree are labeled as "No Collision," with another Gini impurity close to 0.0.

One of the deeper nodes also introduces a condition on the X_car position. This split indicates that in edge cases—when Distance is very close to the threshold—the current position of the car can influence the outcome. For example, if the car is still approaching the obstacle (X_car ≤ 10), it may still be at risk, depending on the precise spatial context. However, this condition affects only a small subset of the data (230 samples), and the resulting leaves have slightly higher Gini values, reflecting some uncertainty or overlap between classes in those cases.

In summary, the decision tree structure confirms that Distance is the primary factor in determining collision risk, with secondary support from X_car in ambiguous cases. The visual clarity of the tree allows human interpreters to verify the model's logic and trust its decision paths—an essential quality for real-time autonomous systems.

4.3.4 Integration into the Lane Change Decision Logic

After training and validating the Decision Tree model, it was integrated into the larger obstacle avoidance pipeline through a Python-based function named predict_lane_change. This function accepts an input vector of five features—X_car, X_obs, Distance, Velocity, and Y_car—and reshapes it into the format required by the model. It then uses the saved decision_tree_obstacle.pkl model to generate a prediction, determining whether a collision is imminent for that given input state.

If the predicted class is 1 (i.e., Collision), the function initiates a steering maneuver by returning a non-zero lane change signal. Specifically, a value of either +1 (change to the right lane) or –1 (change to the left lane) is returned, chosen randomly to simulate dynamic decision-making in multi-lane scenarios. If the prediction is 0 (i.e., No Collision), the function returns a lane change value of 0, instructing the vehicle to continue along its current trajectory. This simple yet effective control policy allows the simulated car to dynamically react to threats based on learned behavior from thousands of previous data points.

The use of a Decision Tree in this context provides multiple advantages. Firstly, the inference time is extremely low, making it suitable for real-time applications such as autonomous

driving. Secondly, the model is interpretable—the tree visualization clearly shows why certain decisions are made, which is essential when debugging safety-critical systems. Finally, the compact nature of the tree (due to controlled max_depth and sample split thresholds) ensures that it generalizes well without overfitting to noise in the training data.

4.3.5 Discussion on Model Robustness and Observations

One of the most important outcomes of this process was the ability to confidently derive collision predictions from simulated sensor data. By leveraging the structured nature of the CSV dataset (covered in Section 4.2), the model was exposed to a wide variety of conditions—including fast- and slow-moving vehicles, distant and nearby obstacles, and different lane positions. This diversity allowed the Decision Tree model to capture both trivial and edge-case behaviors effectively.

The high model accuracy of 99.94% reflects the clean separation between collision and no-collision cases in the training data. This also confirms that the input features selected—particularly Distance and X_car—were appropriate and meaningful for the task. The near-perfect classification at the root node of the tree (with gini = 0.0) emphasizes that the system can decisively react when an obstacle enters the critical range.

Furthermore, because the Decision Tree makes predictions based on simple if-else rules, it integrates smoothly into the existing MATLAB-based simulation loop. Once trained, the model can be called repeatedly at every simulation step, supplying up-to-date decisions based on the evolving state of the vehicle and environment. This is especially useful for embedded systems where resource constraints and explainability are both crucial.

4.4 Graphical User Interface

One of the most critical and visually demonstrative aspects of the obstacle detection and autonomous steering project was the development of a real-time Graphical User Interface (GUI) in MATLAB. This section elaborates on the working of the GUI, the logic incorporated into the lane change behavior, and the visualization of the car's trajectory under different

obstacle-avoidance scenarios. The GUI served not only as a visualization tool but also as a validation platform, providing immediate visual feedback about the behavior of the simulated autonomous vehicle when encountering obstacles.

4.4.1 Objective of GUI Simulation

The primary aim behind developing the MATLAB GUI was to simulate a simplified but dynamic traffic lane environment where the autonomous vehicle (represented by a car icon) encounters random obstacles placed across different lanes. As the simulation progresses, the car detects obstacles based on proximity using trained machine learning models (discussed earlier in Section 4.3) and takes corrective actions such as lane changes to avoid collisions. The simulation closely mirrors the behavior of an autonomous vehicle reacting to real-time LiDAR or radar sensor inputs.

The GUI dynamically updates the position of the car and obstacles as the vehicle moves forward along the X-axis. The car initially starts at an X-position of approximately -5, allowing sufficient room for the environment to develop around it as the simulation progresses. Throughout the simulation, the x-axis of the plot is continuously updated to maintain the car roughly centered within the visible range, stretching from (X_car – 5) to (X_car + 15).

As the car moves forward, obstacles advance leftward relative to the car's frame of reference. Any obstacle that moves closer than 5 units from the extreme left edge of the visible frame is removed and regenerated randomly ahead of the car. This dynamic ensures that the simulation area remains populated with obstacles, maintaining the challenge for the autonomous decision-making system and preserving the continuity of the virtual environment.

4.4.2 Working Principle of the Simulation

The GUI simulation operates on a looped structure where the car moves continuously along the X-axis at a set speed. Obstacles are randomly generated across three discrete lanes, corresponding to Y-positions 1, 2, and 3. As the car advances, it constantly evaluates the distance to the nearest obstacle in its current lane. If an obstacle is detected within a critical threshold distance (less than or equal to 2 units), the Python-trained machine learning model is invoked to predict the collision risk.

Upon detecting a collision, the car initiates a smooth lane change maneuver. Instead of abruptly shifting from one lane to another, the car's Y-coordinate is adjusted incrementally over several steps, resulting in a natural diagonal transition to the adjacent lane. This technique realistically simulates the gradual steering behavior of a physical vehicle.

At each graphical refresh, the obstacles are also repositioned slightly to the left to simulate the continuous forward motion of the environment relative to the car. Obstacles that move out of the frame—i.e., crossing the 5-unit buffer on the left—are regenerated ahead of the car to ensure persistent complexity.

Each frame updates not only the position of the car and obstacles but also provides a context-specific title ("Safe", "Changing Lane", "Lane Change Complete") depending on the stage of motion.

4.4.3 Case 1: Lane Change from Bottom Lane to Middle Lane

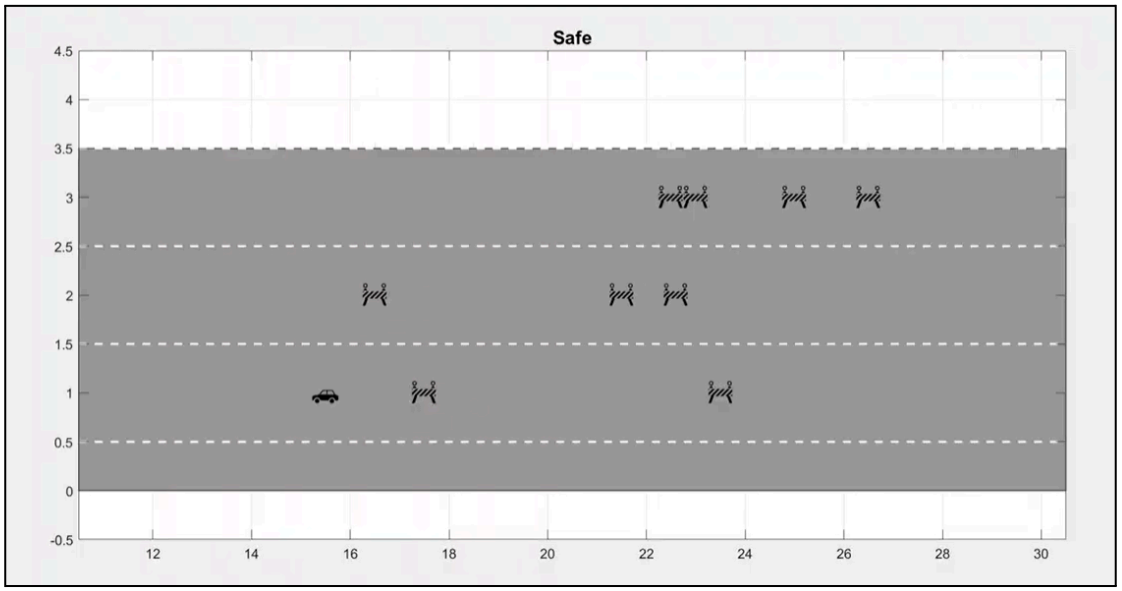


Figure 22. Car running safe in the bottom lane just before Obstacle

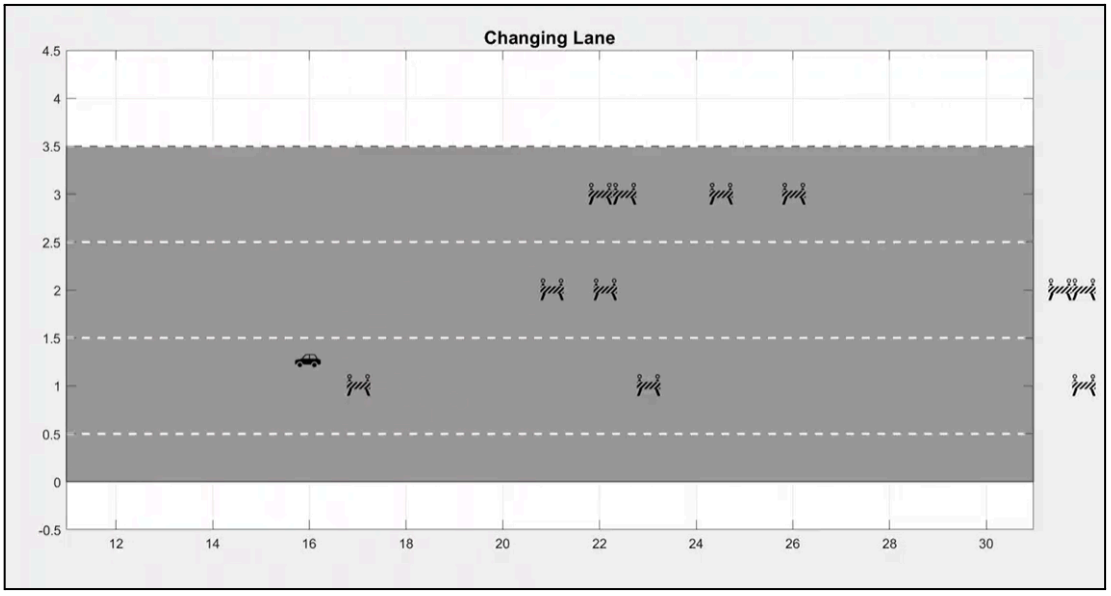


Figure 23. Car changing lane from the bottom lane towards middle lane

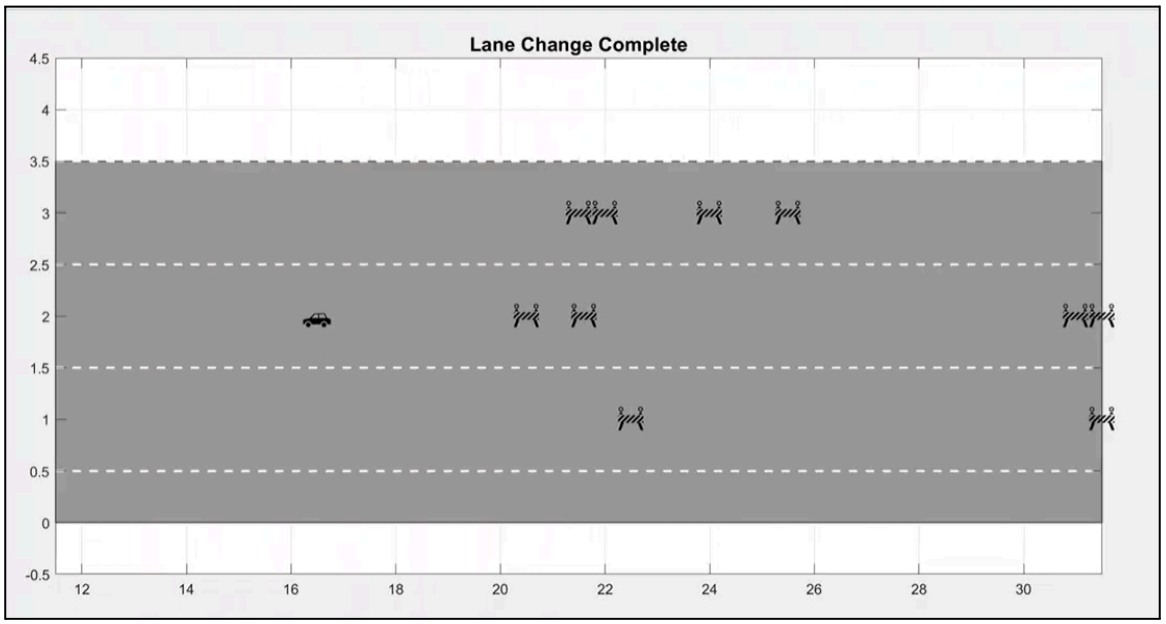


Figure 24. Car running in Middle after lane change

The first lane change scenario corresponds to a transition from the bottom lane (Lane 1, Y ≈ 1) to the middle lane (Lane 2, Y ≈ 2). This sequence occurs at approximately t = 16 seconds of the simulation. Figures 22, 23, 24 depict different phases of this transition.

In Figure 22, titled "Safe", the car is moving forward in the bottom lane with several obstacles detected ahead. The vehicle, at this point, is still at a safe distance from nearby obstacles. However, an obstacle appears within the critical proximity, triggering the collision prediction logic.

In Figure 23, titled "Changing Lane", the car begins its diagonal ascent towards the middle lane. This transition is accomplished by reducing the forward speed by 30% and simultaneously introducing a small, steady vertical velocity component towards the new lane. As a result, the car appears to slide smoothly between the two lanes, mimicking a gentle steering maneuver. The obstacle in the original lane is thus successfully avoided by adjusting the car’s lateral position.

In Figure 24, titled "Lane Change Complete", the car completes its lane change and stabilizes its motion in the middle lane. From this point onward, the car resumes its straight-line forward motion at the standard speed without any further lateral displacement, until the next obstacle triggers another maneuver.

4.4.4 Case 2: Lane Change from Middle Lane to Bottom Lane

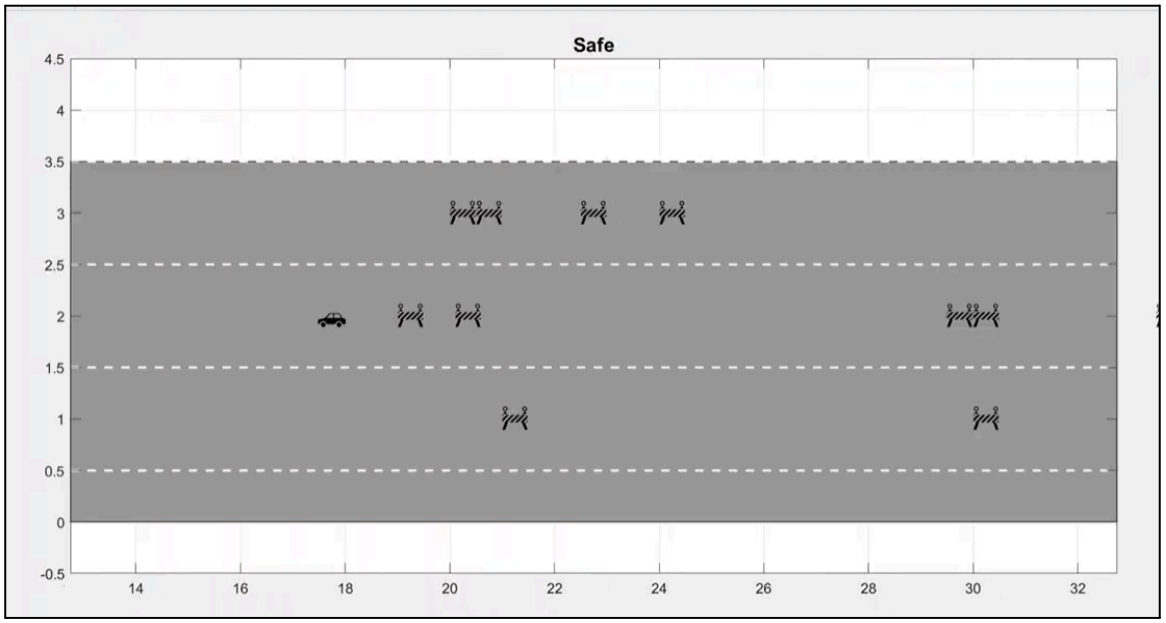


Figure 25. Car running Safe in the Middle Lane just before obstacle

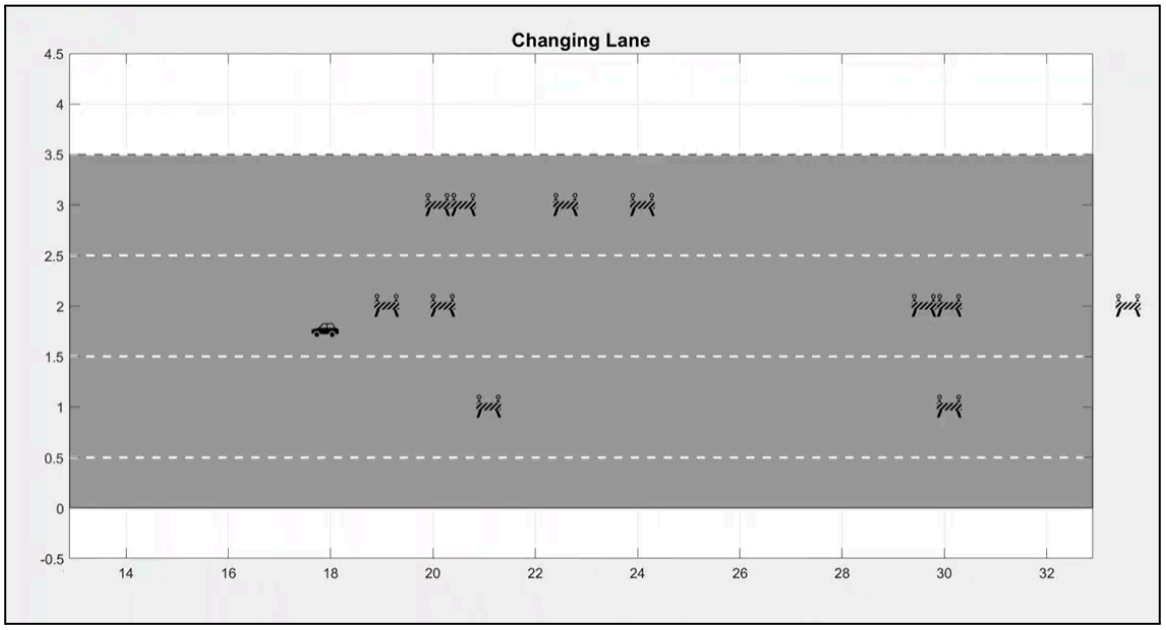


Figure 26. Car changing lane from the middle lane towards the bottom lane

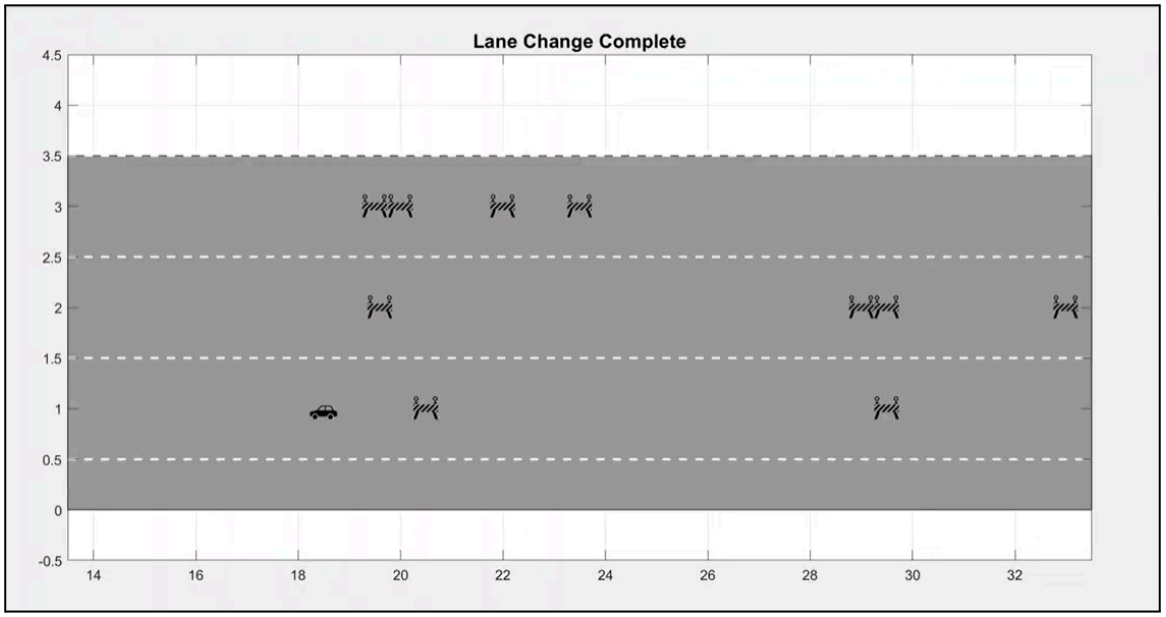


Figure 27. Car running in the bottom lane after lane change

The second lane change event corresponds to the car moving from the middle lane (Lane 2, Y ≈ 2) down to the bottom lane (Lane 1, Y ≈ 1). This transition occurs at approximately t = 18 seconds. Figures 25, 26, 27 illustrate this behavior.

In Figure 25, the car can be seen traveling safely in the middle lane. However, as it moves forward, an obstacle is detected within a dangerously close proximity in the same lane. The collision detection logic, powered by the trained Decision Tree model, correctly predicts the likelihood of collision and initiates a downward lane change maneuver.

In Figure 26, titled "Changing Lane", the car gradually shifts downward towards the bottom lane. As before, the lane transition is achieved by smoothly altering the Y-position across multiple steps, accompanied by a slight reduction in the X-velocity to ensure stability and safety. This deliberate and progressive movement ensures that the vehicle does not experience abrupt or unrealistic changes in trajectory.

By Figure 27, titled "Lane Change Complete", the car has successfully settled into the bottom lane. The transition is complete, and the vehicle resumes its regular forward speed and direction. This case once again demonstrates the flexibility and responsiveness of the system, capable of adapting to obstacle threats by steering either upward or downward as needed.

4.4.5 Case 3: Lane Change from Middle Lane to Top Lane

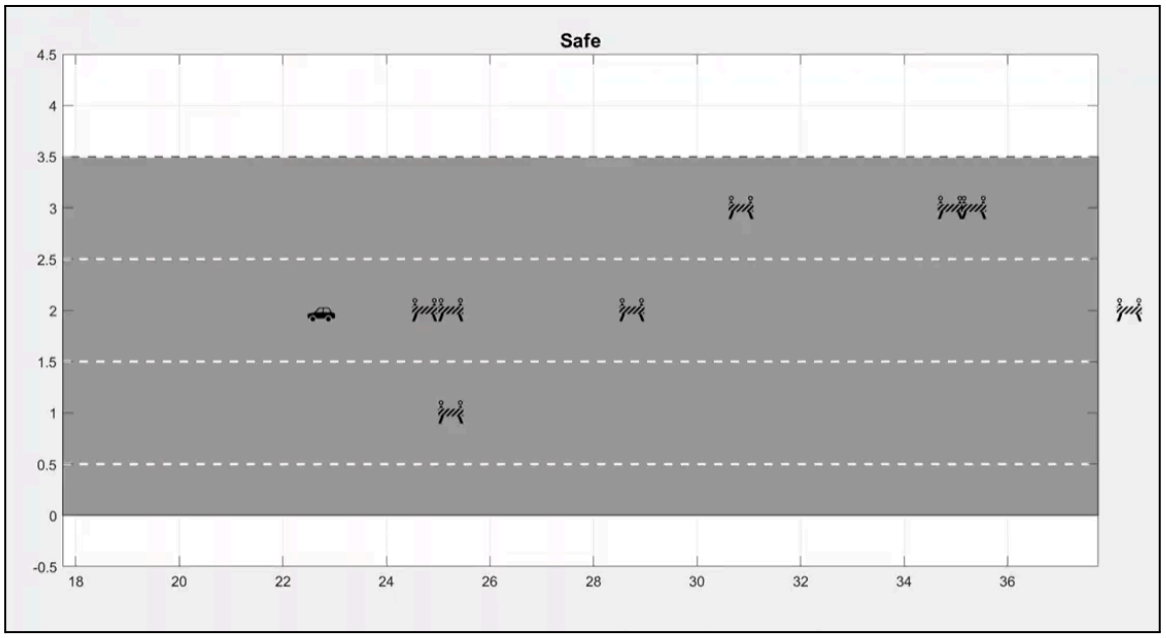


Figure 28. Car running safe in the middle lane just before obstacle

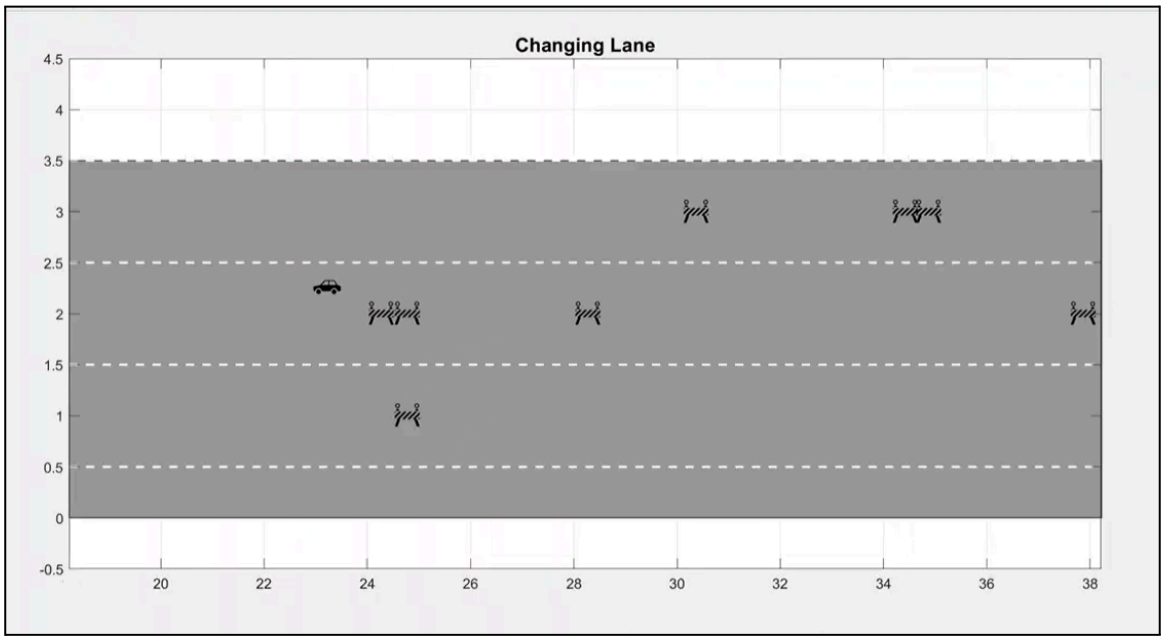


Figure 29. Car changing lane from the middle lane towards top lane

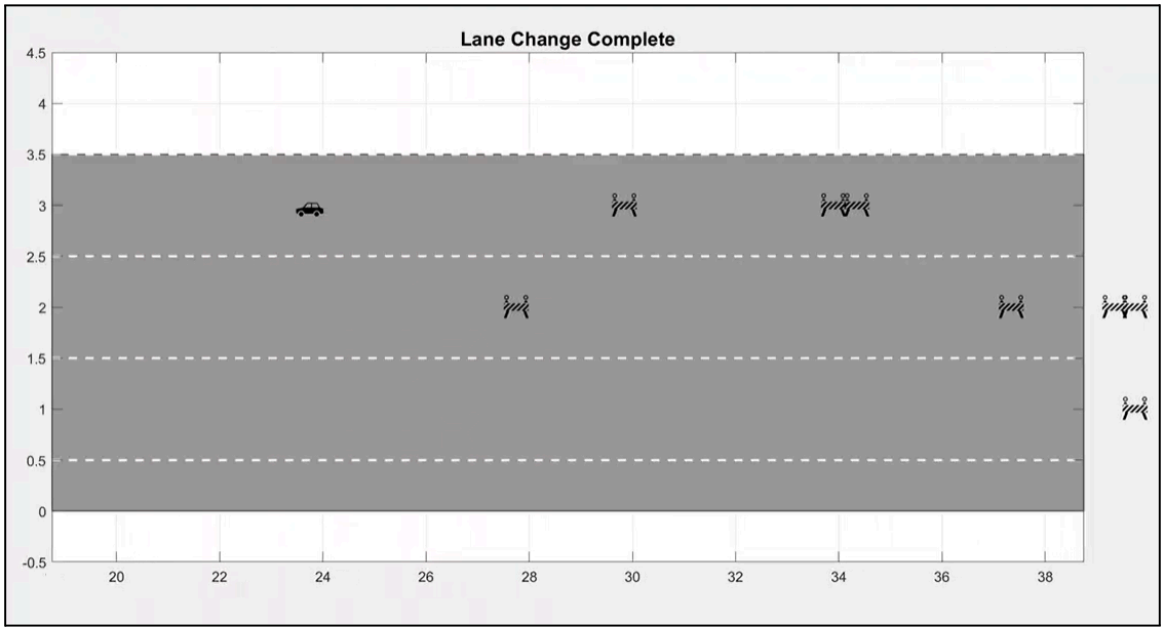


Figure 30. Car running in the top lane after lane change

The third lane change scenario shows a transition from the middle lane (Lane 2, Y ≈ 2) to the top lane (Lane 3, Y ≈ 3), occurring at approximately t = 23 seconds. Figures 28, 29, 30 capture this sequence.

In Figure 28, the vehicle is moving forward in the middle lane under safe conditions. However, as the car progresses, a new obstacle is detected within the critical distance. The predictive model responds by determining that a collision is imminent if no evasive action is taken.

Figure 29, labeled "Changing Lane", shows the car executing an upward lane change maneuver. The steering behavior follows the same principle of incremental Y-velocity addition and a 30% temporary speed reduction in the X-direction, producing a gentle diagonal transition between lanes. This realistic behavior closely mirrors actual vehicle dynamics during lane changing operations.

By Figure 30, the car has fully moved into the top lane, and the title changes to "Lane Change Complete". At this point, the car continues its forward journey at regular speed, having successfully avoided the obstacle without sudden or jerky motions.

4.4.6 Case 4: Lane Change from Top Lane to Middle Lane

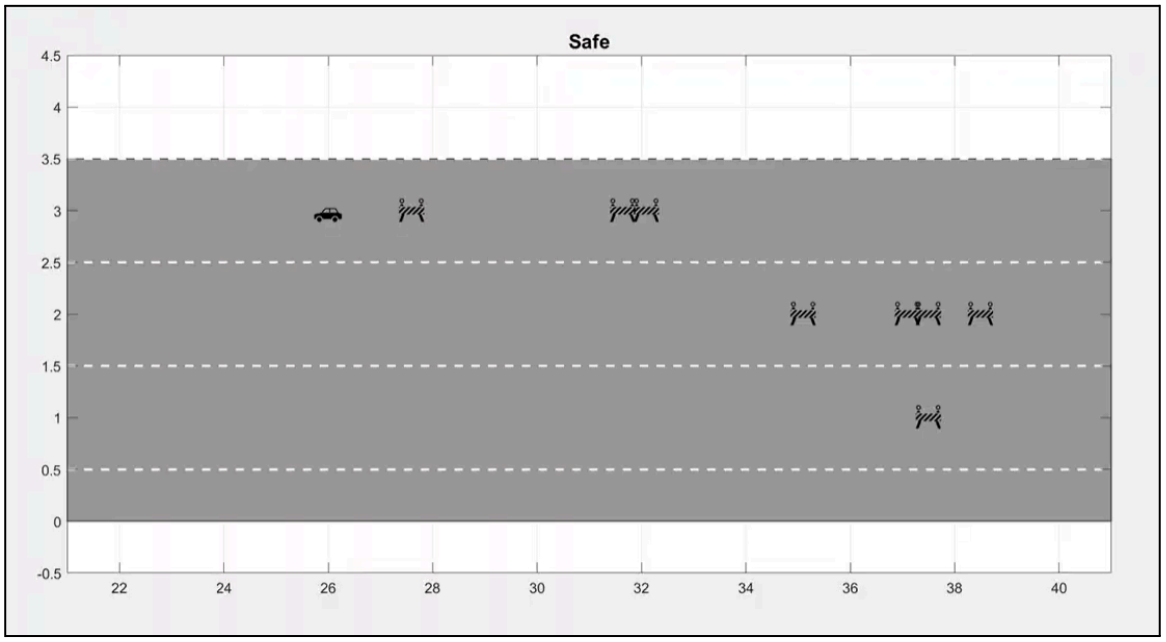


Figure 31. Car running safe in the top lane just before obstacle

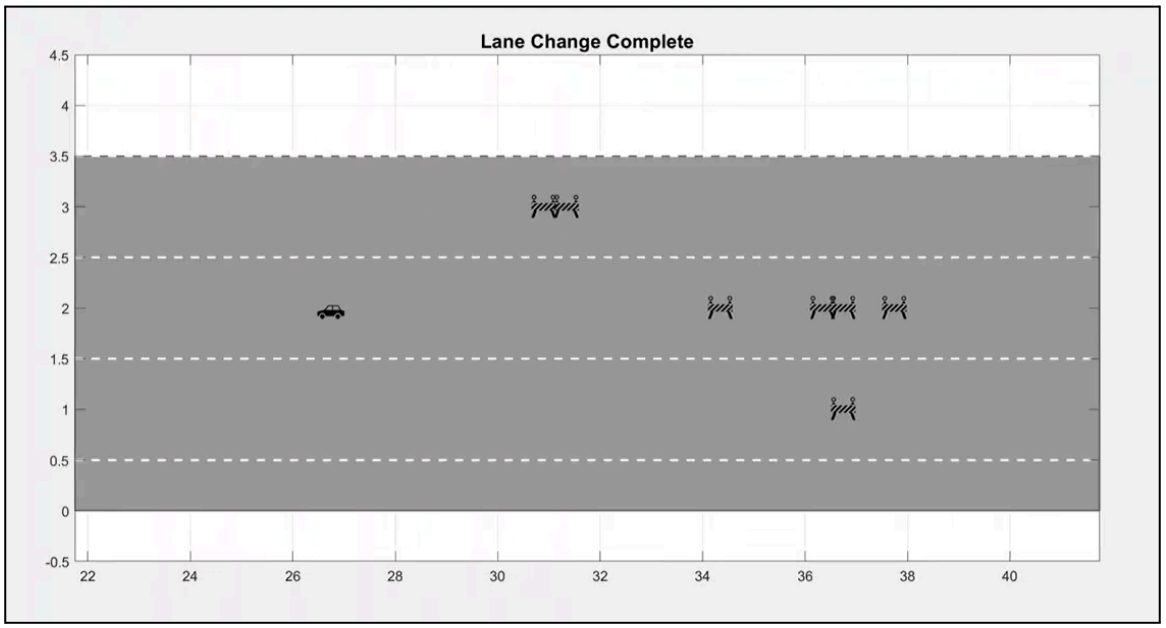


Figure 32. Car changing lane from the top lane towards the middle lane

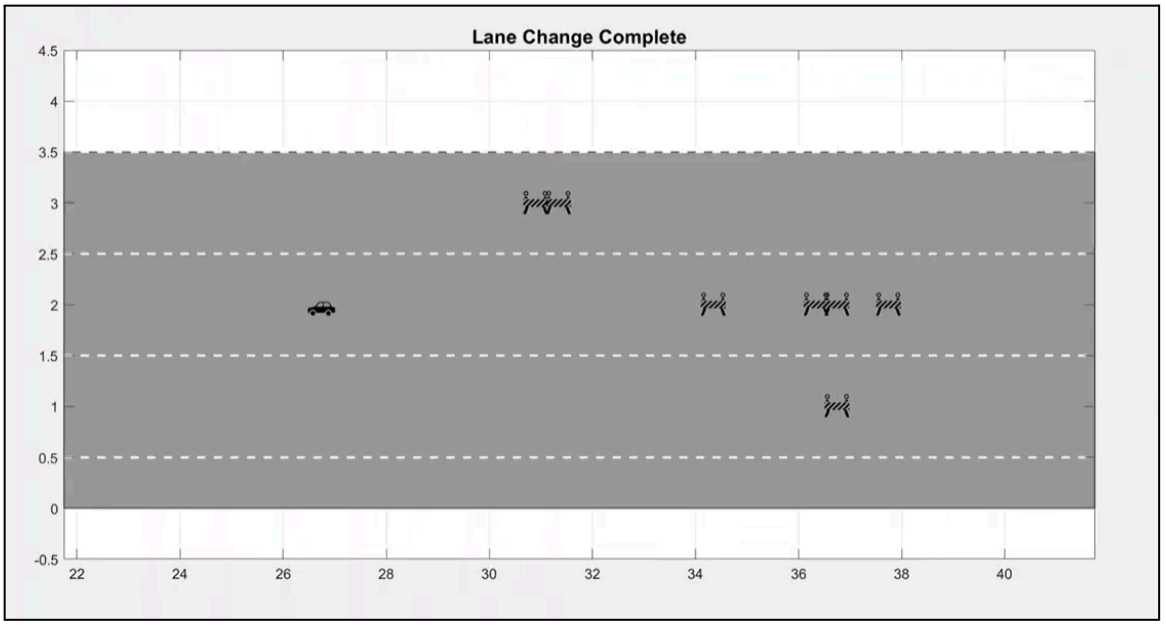


Figure 33. Car running in the middle lane after lane change

The final lane change corresponds to a transition from the top lane (Lane 3, Y ≈ 3) down to the middle lane (Lane 2, Y ≈ 2), happening at approximately t = 26 seconds. Figures 31, 32, 33 represent this event.

In Figure 31, the car is seen traveling along the topmost lane. The system detects an obstacle directly in its path, activating the collision prediction mechanism. As before, the response is quick and calculated.

Figure 32 shows the car entering the "Changing Lane" phase, diagonally descending towards the middle lane. The smooth trajectory once again affirms the efficacy of the implemented steering logic that leverages fine-grained Y-coordinate adjustments over multiple steps.

Finally, in Figure 33, the "Lane Change Complete" state is achieved, with the car now safely positioned in the middle lane. Normal motion resumes, and the simulation continues to dynamically react to the evolving environment.

4.4.7 Summary of GUI

The Graphical User Interface (GUI) developed for this project successfully replicates the dynamics of an autonomous vehicle navigating through a simulated multi-lane road environment populated with obstacles. Through real-time visual updates, the GUI captures both safe straight-line travel and complex lane change maneuvers triggered by collision predictions. By introducing incremental lane transitions combined with controlled speed reduction, the system realistically mimics the behavior of a physical autonomous vehicle responding to environmental challenges.

Across the four distinct cases discussed — bottom to middle, middle to bottom, middle to top, and top to middle lane changes — the simulator consistently demonstrates its ability to detect imminent threats and steer accordingly. The seamless integration between MATLAB's environment, real-time decision-making powered by Python's machine learning models, and dynamic visualization reaffirms the strength of the cross-platform approach adopted in this work.

Overall, the GUI not only enhances understanding of obstacle avoidance behavior but also acts as a powerful validation tool for the underlying machine learning and control algorithms, bridging the gap between simulation and potential real-world deployment.

CHAPTER 5: CONCLUSION & FUTURE SCOPES

5.1 Conclusion & Summary

The mini project titled “Obstacle Detection and Automated Steering using Python and MATLAB Simulink” successfully achieves its goal of building a simulation-based intelligent driving system that reacts to environmental threats in real-time. The project integrates multiple disciplines including control systems, machine learning, and graphical simulation to demonstrate how an autonomous vehicle could make steering decisions based on sensed obstacle proximity.

A major strength of this project lies in its data-driven approach. Using a Simulink model, a large dataset of 168,449 labeled samples was generated under varying conditions of obstacle position and car velocity. This dataset captured a wide range of real-time dynamics including time-series values for the car's position, velocity, obstacle proximity, and collision flags. From this data, a Decision Tree classifier was trained using Python and achieved a remarkably high accuracy of 99.94%. This strong performance was validated using classification metrics and visual analysis of the trained tree structure, confirming the model’s ability to separate collision and non-collision cases with high confidence.

The Results and Discussion chapter provided a detailed breakdown of both intermediate and final outcomes. The initial collision detection scope output demonstrated that the model reliably detects obstacle presence based on distance thresholds. The second output, an XY graph, clearly illustrated how the vehicle deviates from its path in response to a predicted collision. Notably, the project implemented smooth steering transitions rather than abrupt lane jumps. The vehicle’s motion was governed by both X- and Y-axis velocities, and speed was reduced during lane change to reflect realistic maneuvering.

Additionally, four specific lane change scenarios were simulated and visualized in the Graphical User Interface (GUI): bottom-to-middle, middle-to-bottom, middle-to-top, and top-to-middle. Each of these transitions was triggered at a particular timestamp, reinforcing

the correctness of the logic. Figures corresponding to these moments demonstrated the effectiveness of the steering response over time, and the GUI consistently updated the environment to reflect car movement, obstacle regeneration, and lane change state. The dynamic behavior of the vehicle, along with scene labels like “Safe”, “Changing Lane”, and “Lane Change Complete”, made the system intuitive to interpret.

Another key contribution was the real-time integration of Python and MATLAB. The trained model was saved as a .pkl file and loaded directly into MATLAB using pyenv and system interface, enabling live predictions. This tight coupling between predictive logic and vehicle dynamics ensured that machine learning decisions were not only accurate but also instantly executable within the simulation loop.

In summary, this project demonstrates a successful implementation of a miniaturized autonomous control system using machine learning and simulation. The collision detection mechanism, intelligent steering logic, and GUI visualization work together to replicate realistic driving behavior. The modular nature of the system allows for easy extension into more complex scenarios involving dynamic obstacles, multiple vehicles, or even deployment on embedded hardware. Most importantly, the structured experimentation and discussion of results confirm that the project not only meets its technical objectives but also serves as a strong proof-of-concept for further exploration in intelligent transportation systems.

5.2 Future Scope

One significant direction for future development is the physical realization of the simulated model. The system can be translated into a scaled hardware prototype, such as an autonomous RC car equipped with real sensors like LiDAR, ultrasonic rangefinders, and inertial measurement units (IMUs). The same obstacle detection logic, trained Decision Tree model, and steering control algorithms can be deployed onto an embedded controller, such as a Raspberry Pi, NVIDIA Jetson, or a dedicated automotive microcontroller. Real-time sensor data would replace the virtual inputs used in the current simulation, offering a more dynamic and challenging environment for testing.

Another promising scope is the inclusion of driver behavior emulation. While the current system operates in a fully autonomous mode, future iterations could integrate modules for interpreting human driver inputs, such as steering wheel commands, brake pedal pressure, and throttle application. Machine learning models could then be trained not only to react to obstacles but also to adaptively assist or override human inputs when necessary, leading toward the development of advanced driver-assistance systems (ADAS).

Moreover, enhancing the obstacle environment to include moving obstacles, varying weather conditions (such as rain or fog), and complex urban scenarios would significantly increase the robustness of the system. Deep learning methods like Convolutional Neural Networks (CNNs) or Reinforcement Learning could be explored to create models that not only detect but also predict obstacle behavior, allowing for anticipatory steering decisions.

REFERENCES

[1] SAE International. (2018). Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles. J3016_201806.

[2] https://www.cardekho.com accessed on 14th March, 2025

[3] Ziegler, J., et al. (2014). "Making Bertha Drive—An Autonomous Journey on a Historic Route." IEEE Intelligent Transportation Systems Magazine, 6(2), 8–20.

[4] https://stock.adobe.com/in/search/images?k=lidar accessed on 14th March, 2025

[5] Ackermann, J., et al. (2002). "Robust Control: The Parameter Space Approach." Springer, Chapters on Vehicle Steering Control and Lateral Dynamics.

[6] https://en.wikipedia.org/wiki/Simulink accessed on 24th March, 2025

[7] https://en.wikipedia.org/wiki/Python_(programming_language) accessed on 24th March, 2025

[8] Fasoli Mauri – "Obstacle Avoidance Strategies for Omnidirectional Mobile Platforms" (2021)

[9] Giancarlo Pacitti – "Automated Driving System Design and Simulation Using MATLAB and Simulink" (2019)

[10] Asif Ahmed – "Obstacle Avoidance System for Autonomous Driving" (2022)

[11] Fernando Castaño et al. – "Obstacle Recognition Based on Machine Learning for On-Chip LiDAR Sensors in a Cyber-Physical System" (2017)